

STM32 Inbetriebnahme

Author: Pietro Pagliarulo

Version: 1.1

Datum: 17.04.2025

Inhalt

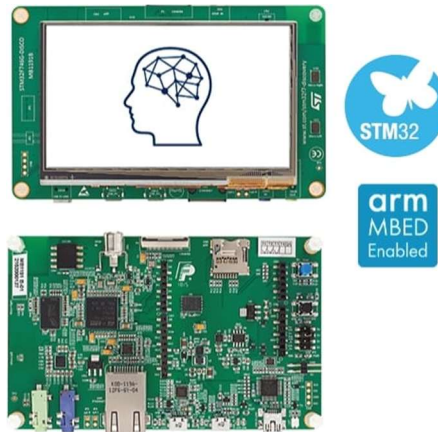
1	Das Board STM32F746G-DISCO	3
1.1	HW-Revision vom Board und Schematic	4
1.2	Verendeter Mikrokontroller	5
2	Getting started Webseite	6
3	Software Development Tools	6
3.1	STM32CubeIDE installieren.....	6
3.1.1	MyST Account anlegen	7
3.2	Installation von STM32CubeMX.....	8
3.3	Installation Treiber, Demos und Beispiele.....	8
3.4	STM32CubeIDE starten.....	10
3.4.1	Programmer für IDE aktualisieren.....	10
3.4.2	Programmierungsumgebung testen (Workspace: ExampleTest)	11
3.5	LED1 Projekt.....	15
3.6	X-CUBE-TouchGFX installieren.....	22
3.6.1	Install Programmer.....	22
3.7	X-CUBE-TouchGFX starten und ein erstes Projekt entwickeln.....	23

1 Das Board STM32F746G-DISCO

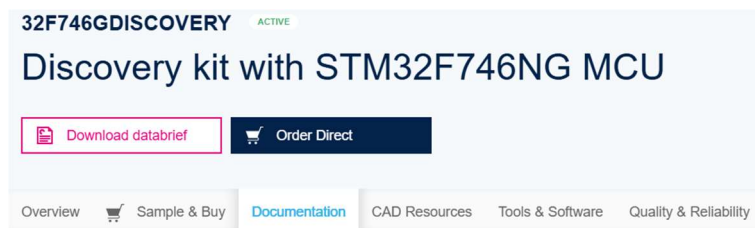
Das Board STM32F746G-DISCO ist eine vollständige Demonstration- und Entwicklungsplattform für STMicroelectronics Arm Cortex M7 Mikrokontroller.

Die technische Beschreibung von dem Board ist unter folgendem Link zu finden:

[32F746GDISCOVERY - Discovery kit with STM32F746NG MCU - STMicroelectronics](#)



Auf der Webpage mit der technischen Beschreibung sind verschiedene Dokumente zugänglich:



In dem Bereich „Documentation“ befindet sich die Produktspezifikation „32f746gdiscovery.pdf“ und andere nützliche Bedienungsanleitungen:

User Manuals (3)

	UM1907 Discovery kit for STM32F7 Series with STM32F746NG MCU	5.0	13 Jan 2020
	UM2052 Getting started with STM32 MCU Discovery Kits software development tools	2.0	25 Feb 2019
	UM1969 Getting started with STM32F746G discovery software development tools	1.0	01 Dec 2015

Die Datei

„um1907-discovery-kit-for-stm32f7-series-with-stm32f746ng-mcu-stmicroelectronics.pdf“ beschreibt das Hardware Layout und die Konfiguration von dem Board und beschreibt auch die vorhandenen Schnittstellen.

In dieser Datei sind auch die verschiedenen Entwicklungsumgebungen präsentiert (Keil MDK-ARM, IAR EWARM, GCC-based IDEs, Arm Mbed, ...).

Figure 2. Hardware block diagram

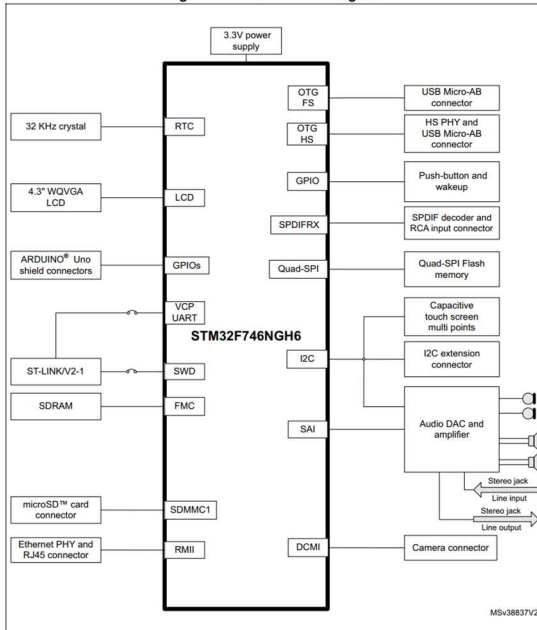
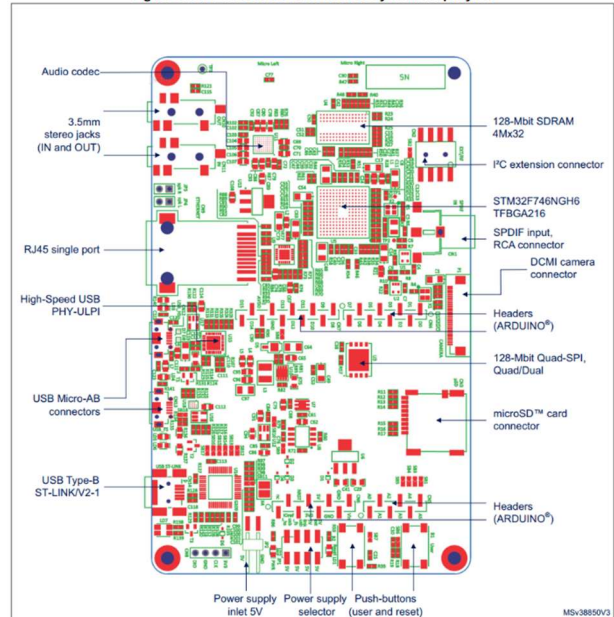


Figure 3. 32F746GDISCOVERY Discovery board top layout



Das Board enthält auch eine Programmierung/Debug-Schnittstelle (USB Type-B ST-LINK/V2-1), die verwendet wird, um das Board an einer PC-Programmierungsumgebung anzuschließen.

1.1 HW-Revision vom Board und Schematic

Das Board ist in verschiedenen HW-Revisionen (BXX, CXX) gebaut worden. Auf der Platine ist die Revision vermerkt:



In dem Bild ist hier beispielsweise die HW-Revision C01 zu sehen. Auf der Webpage mit der technischen Beschreibung befindet sich unter dem Bereich „CAD Resources“ die Liste der Schematics der verschiedenen HW-Revisionen:

Schematic Pack (4)

	MB1191-F746NGH6-B02 Board Schematic	1.0	10 Feb 2020
	MB1191-F746NGH6-C01 Board Schematic	2.0	10 Feb 2020
	MB1191-F746NGH6-C02 Board Schematic	1.0	06 Dec 2022
	MB1191-F746NGH6-C03 Board Schematic	1.0	06 Dec 2022

Das Schematic der entsprechenden HW-Revision soll heruntergeladen werden (hier in dem Beispiel C01).

1.2 Verendeter Mikrokontroller

Der verwendete Mikrokontroller ist das **STM32F746NG** (STM 32 F7 46NG**H6** Bestellnummer).

STM: STMicroelectronics

32: 32 Bit Mikrokontroller

F7: Arm Cortex M7

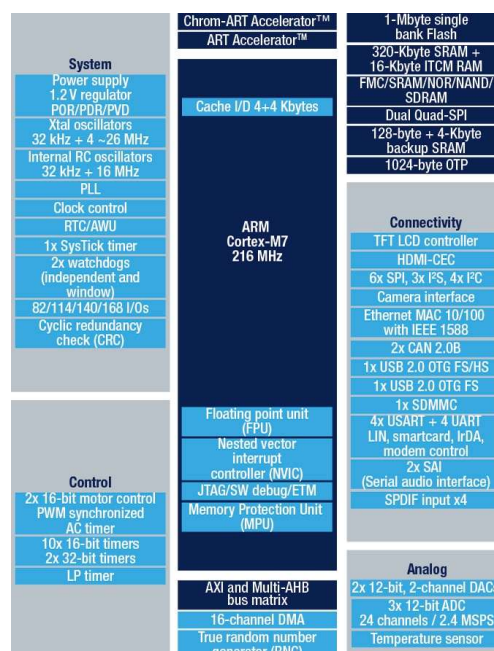
46N: das Mikrokontroller-Derivat: mit LCD-TFT Schnittstelle, 216 MHz CPU, TFBGA216 Package

G: 1 Mbyte Flash Speicher

H6: Index für die Bestellung (Packing Type: Tray)

Die Dokumentation des Mikrokontrollers ist unter folgendem Link zu finden:

[STM32F746NG - High-performance and DSP with FPU, Arm Cortex-M7 MCU with 1 Mbyte of Flash memory, 216 MHz CPU, Art Accelerator, L1 cache, SDRAM, TFT - STMicroelectronics](#)



Auf der Webseite mit der Dokumentation des Mikrokontrollers befinden sich unterschiedliche Dokumente, die nach Bedarf heruntergeladen werden können.

Product Specification, Reference Manual und Programming Manual sollten normalerweise zum Nachschlagen zur Verfügung stehen:

Product Specifications (1)

	DS10916	ARM®-based Cortex®-M7 32b MCU+FPU, 462DMIPS, up to 1MB Flash/320+16+ 4KB RAM, USB OTG HS/FS, ethernet, 18 TIMs, 3 ADCs, 25 com if, cam & LCD	4.0	18 Feb 2016
---	---------	--	-----	-------------

Reference Manuals (1)

	RM0385	STM32F75xxx and STM32F74xxx advanced Arm®-based 32-bit MCUs	8.0	09 Jul 2018
---	--------	---	-----	-------------

Programming Manuals (1)

	PM0253	STM32F7 Series and STM32H7 Series Cortex®-M7 processor programming manual	5.0	20 Jun 2019
---	--------	---	-----	-------------

2 Getting started Webseite

Hilfreich für die erste Inbetriebnahme ist folgende Webseite nützlich:

[Getting started with STM32: STM32 step-by-step - stm32mcu](#)

Dort findet man allgemeine Informationen über den Mikrokontroller und über die Programmierumgebung, einschließlich Beispiele für erste Inbetriebnahme (aber mit einem anderen Board).

3 Software Development Tools

Für die STM32 gibt es zahlreiche Development Tools mit unterschiedlichen Eigenschaften/Komplexität und Kosten.

Hier wird auf die folgenden Development Tools eingegangen:

- STM32CubeF7 (für die Standard Programmierung)
- X-CUBE-TouchGFX (für die Programmierung von grafischen Benutzerschnittstellen)

Die Software Tools sind auf der Webpage mit der technischen Beschreibung des Boards in dem Bereich „Tools & Software“ zugänglich:

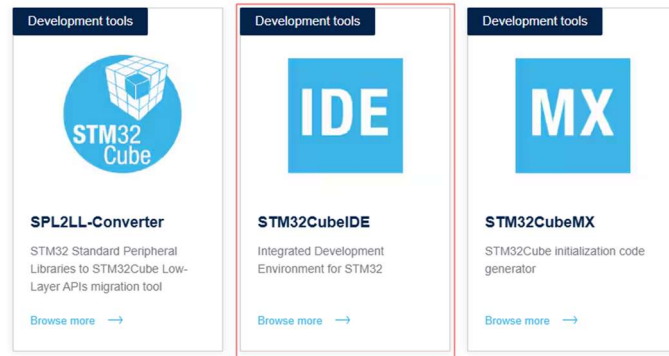
The screenshot shows the product page for the STM32F746GDISCOVERY kit. The main heading is 'Discovery kit with STM32F746NG MCU'. Below this are buttons for 'Download databrief' and 'Order Direct'. A navigation bar includes 'Overview', 'Sample & Buy', 'Documentation', 'CAD Resources', 'Tools & Software' (highlighted with a red box), and 'Quality & Reliability'. Below the navigation bar is a table of software packages.

Part number	Status	Description	Type	Supplier
STM32CubeF7	ACTIVE	STM32Cube MCU Package for STM32F7 series (HAL, Low-Layer APIs and CMSIS, USB, TCP/IP, File system, RTOS, Graphic - and examples running on ST boards)	STM32Cube MCU & MPU Packages	ST
X-CUBE-AZRTOS-F7	ACTIVE	Azure RTOS software expansion for STM32Cube for STM32F7 series	STM32Cube Expansion Packages	ST
X-CUBE-TOUCHGFX	ACTIVE	TouchGFX advanced and free of charge graphical framework optimized for STM32 microcontrollers	STM32Cube Expansion Packages	ST

Beim Klicken auf STM32CubeF7 erscheint die Beschreibung der Tools und der Bereich für das Download der Treiber und Beispiele.

3.1 STM32CubeIDE installieren

In dem Bereich “Tools & Software” die Software STM32CubeIDE für das gewünschte Betriebssystem installieren:



Get Software

	Part Number ▲	General Description	Latest version ↕	Download	All versions ↕
+	STM32CubeIDE-DEB	STM32CubeIDE Debian Linux Installer	1.16.0	Get latest	Select version ▼
+	STM32CubeIDE-Lnx	STM32CubeIDE Generic Linux Installer	1.16.0	Get latest	Select version ▼
+	STM32CubeIDE-Mac	STM32CubeIDE macOS Installer	1.16.0	Get latest	Select version ▼
+	STM32CubeIDE-RPM	STM32CubeIDE RPM Linux Installer	1.16.0	Get latest	Select version ▼
+	STM32CubeIDE-Win	STM32CubeIDE Windows Installer	1.16.0	Get latest	Select version ▼

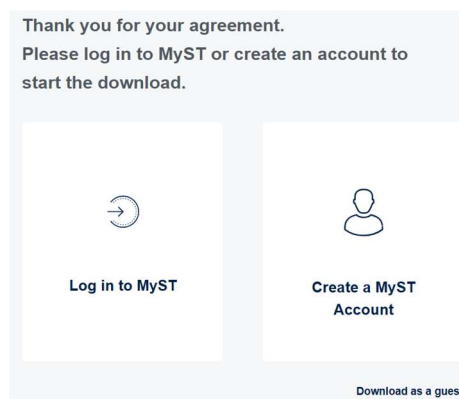
Für Windows ist die heruntergeladene Datei in .exe Format und erfordert Admin-Rechte für die Installation von der IDE.

Nach dieser Installation steht die IDE als Shortcut auf dem Desktop zur Verfügung (z.B.):



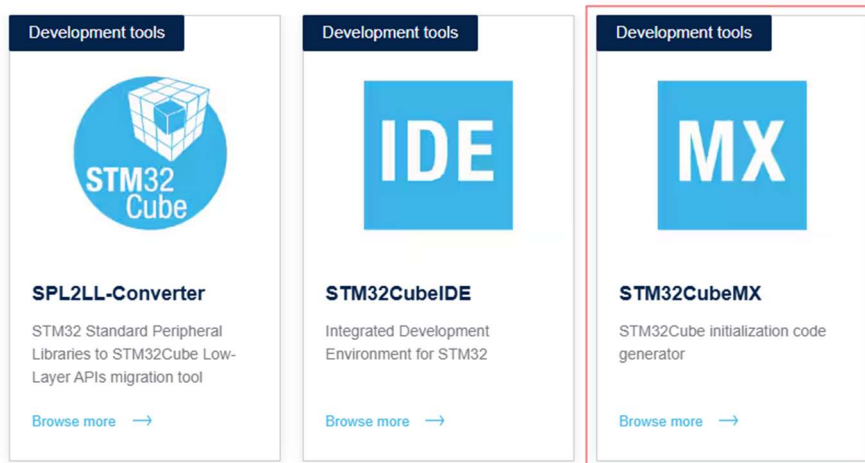
3.1.1 MyST Account anlegen

Für das Download ist notwendig, ein MyST Account zu haben (wichtig auch für alle andere Aktivitäten während der Entwicklung)



Bitte auf „Create a MyST Account“ klicken und die Anweisungen folgen.

3.2 Installation von STM32CubeMX



STM32CubeMX installieren. Diese SW wird benötigt, um die Peripherie des Mikrokontrollers visuell zu konfigurieren. Auch hier muss die Version entsprechend Betriebssystem heruntergeladen.

Eine Zip Datei wird heruntergeladen. Beim Entpacken entsteht ein .exe Datei mit dem das Tool installiert wird.

3.3 Installation Treiber, Demos und Beispiele

Auf der Webseite [STM32CubeF7 - STM32Cube MCU Package for STM32F7 series \(HAL, Low-Layer APIs and CMSIS, USB, TCP/IP, File system, RTOS, Graphic - and examples running on ST boards\) - STMicroelectronics](#)

Weiter nach unten scrollen und in dem Bereich „Get Software“ die Treiber und Beispiele herunterladen:

Get Software

Part Number ▲	General Description	Latest version ⚙	Download	All versions ⚙
+ Patch_CubeF7	Patch for STM32CubeF7	1.17.2	Get latest	Select version ▼
+ STM32CubeF7	STM32Cube MCU Package for STM32F7 series	1.17.0	Get latest Get from GitHub	Select version ▼

Auf dem Link der Hauptanwendung (hier im Bild 1.17.0) auf „Get latest“ klicken und Lizenzvereinbarungen lesen und akzeptieren.

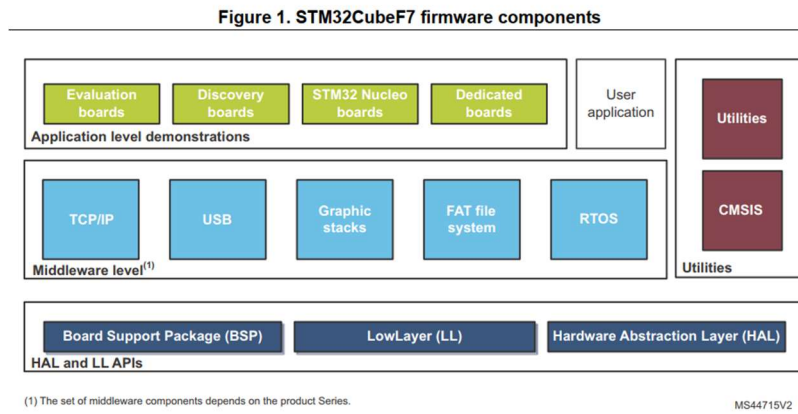
Die Installationsdateien sind in einem Zip-Datei enthalten (z.B. en.stm32cubef7_v1-17-0.zip). Zip Datei entpacken.

Je nach Bedarf eventuell vorhandene Patches herunterladen und installieren (z.B. Patch for STM32CubeF7). Einige Dateien im Zielordner werden überschrieben und auf die letzte korrigierte Version aktualisiert.

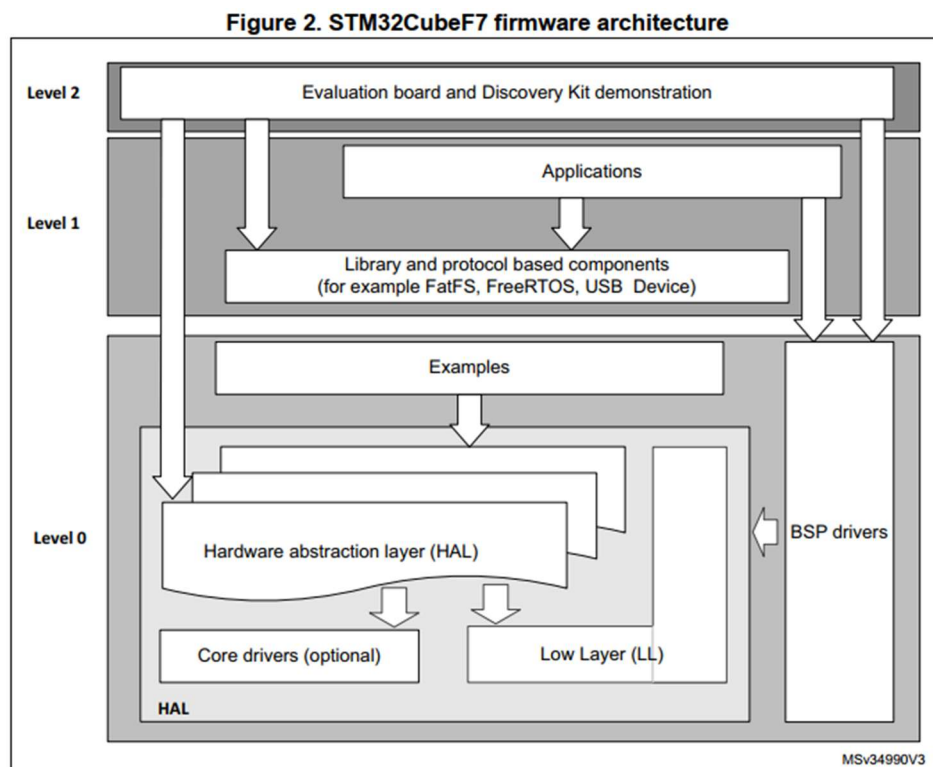
Jetzt stehen die Beispiele und Treiber in dem entpackten Ordner zur Verfügung (Beispiel: Ordner STM32Cube_FW_F7_V1.17.0).

In der entpackten Ordnerstruktur, die Datei „STM32CubeF7GettingStarted.pdf“ in dem Ordner „Documentation“ aufmachen.

In der Datei sind die Grundprinzipien der SW-Demonstrationen beschrieben.



Es ist wichtig, mit dem Konzept der Firmware Architektur sich vertraut zu machen.

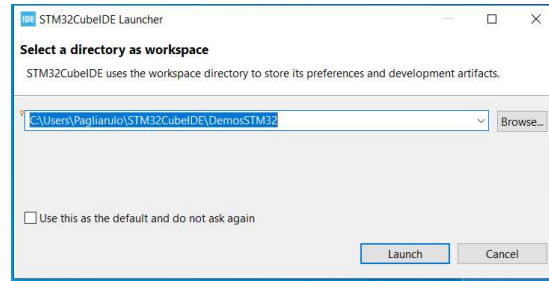


Board support package (BSP)
 Hardware abstraction layer (HAL)
 Basic peripheral usage examples

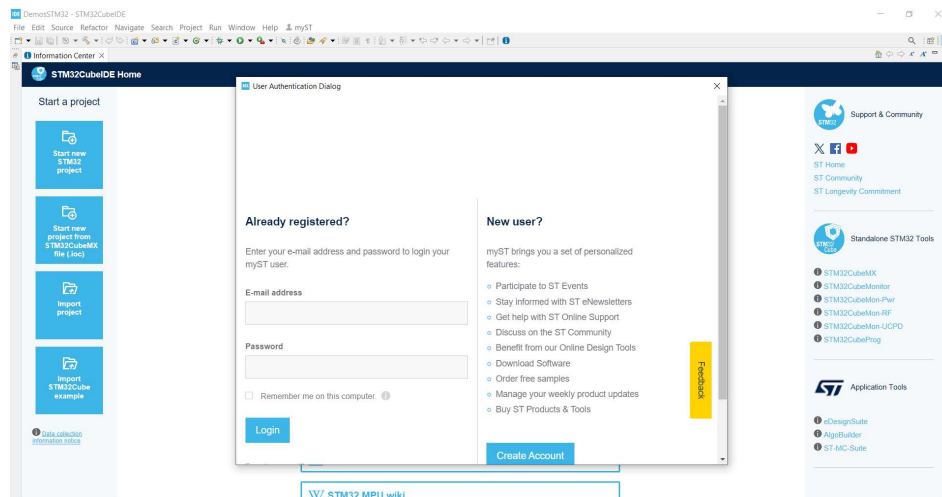
3.4 STM32CubeIDE starten

Die STM32CubeIDE starten indem man den Shortcut auf dem Desktop klickt.

Der Pfad zu einem Workspace muss zwingend definiert werden, damit das Tool starten kann.

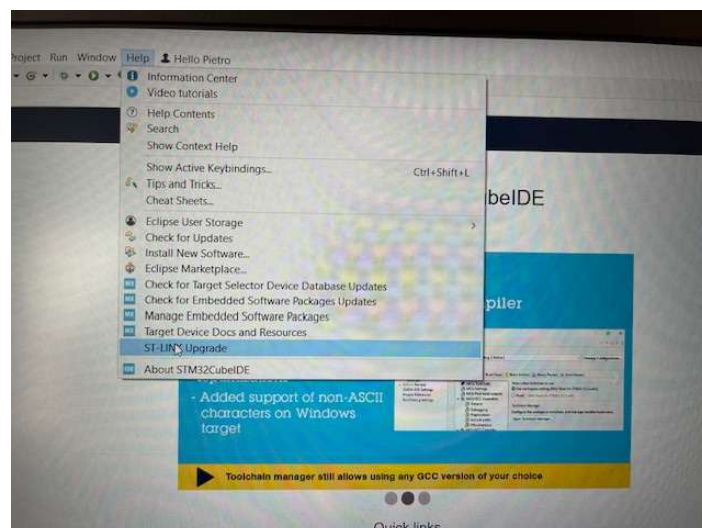


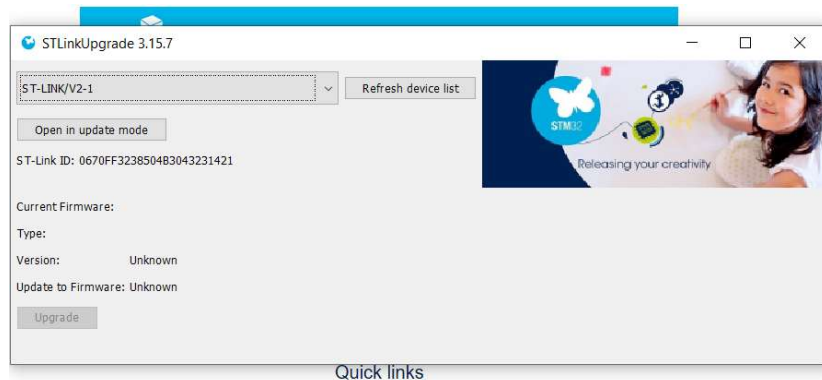
Auf dem Menu myST sich mit E-Mail Adresse und Password anmelden und angemeldet bleiben.



3.4.1 Programmier für IDE aktualisieren

Auf dem Menu Help, auf „ST-LINK Upgrade“ klicken und dann “open in update mode” und auf „Upgrade“ klicken.





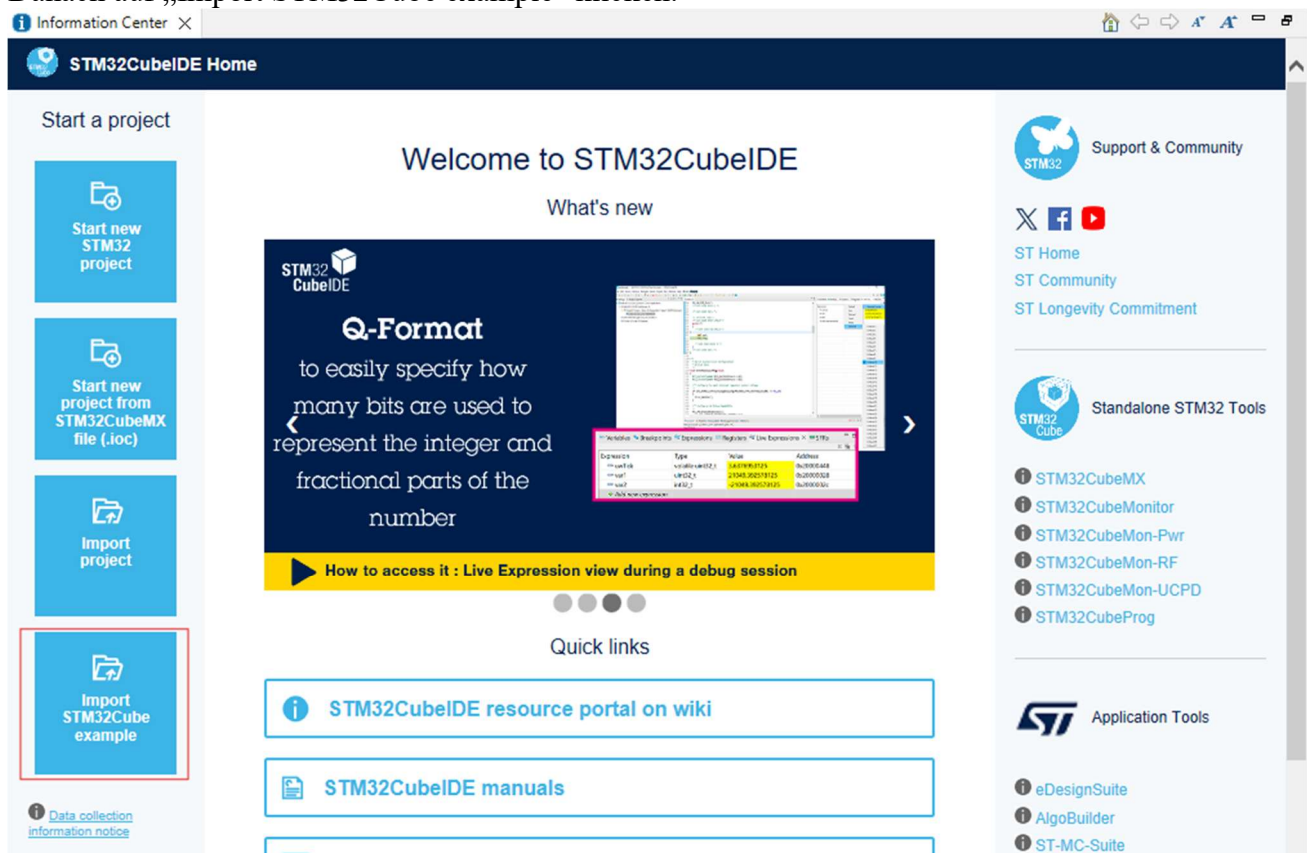
Somit wird die Firmware für Programmierung/Debug-Schnittstelle aktualisiert (sorgt für mehr Stabilität in der Verwendung)

3.4.2 Programmierungsumgebung testen (Workspace: ExampleTest)

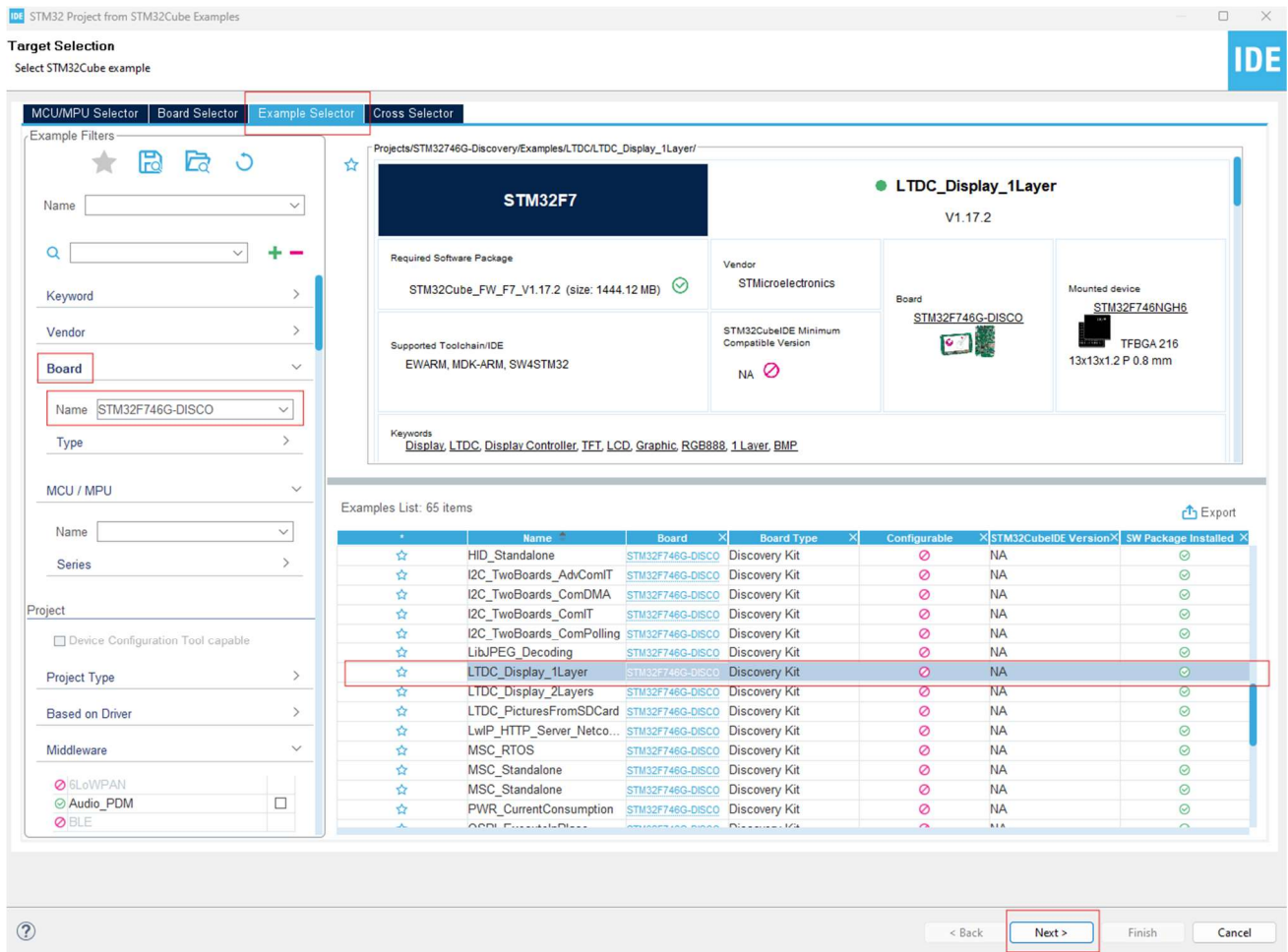
Um sicherzustellen, dass die Programmierungsumgebung funktioniert, kann man ein Beispiel für das Board verwenden.

STM32CubeIDE starten und ein Workspace definieren (besser ein Workspace pro Projekt anlegen).

Danach auf „Import STM32Cube example“ klicken:

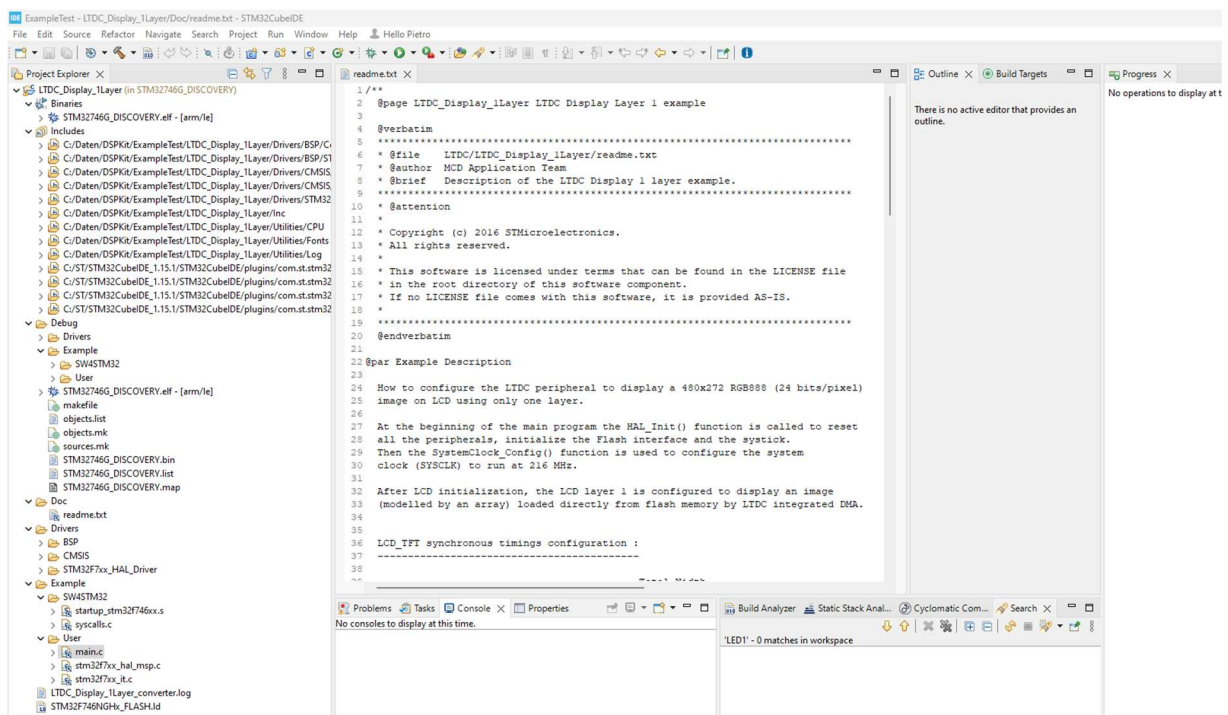


Das Beispiel „LTDC_Dispaly_1Layer“ selektieren indem in dem „Example Selector“ das Board „STM32F746G-Disco“ auswählt und das Beispiel „LTDC_Dispaly_1Layer“ in der Exmaple List auswählt. Dann auf Next klicken:

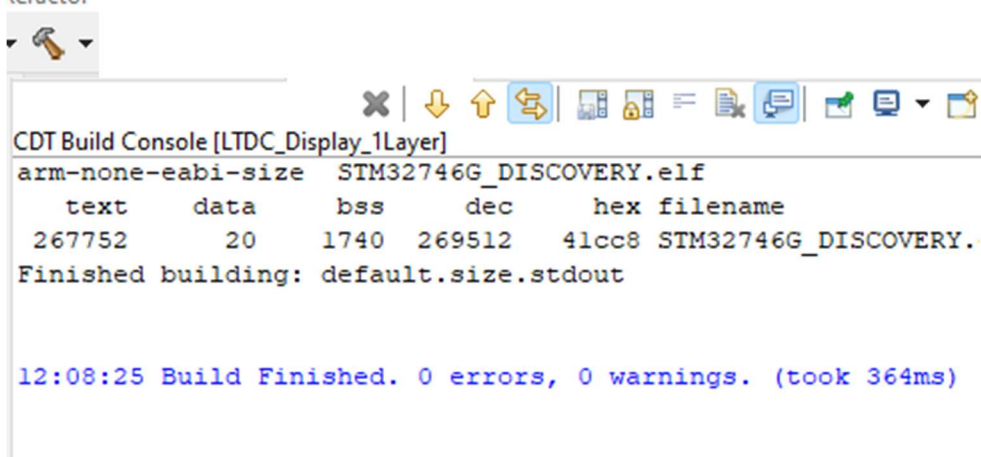


Alle anderen Parameter in den nächsten Menu so lassen, wie voreingestellt und auf Finish klicken.

Folgende Umgebung (Eclipse) sollte sichtbar sein:



Projekt Builden:



```
CDT Build Console [LTDC_Display_1Layer]
arm-none-eabi-size STM32746G_DISCOVERY.elf
  text    data    bss     dec     hex filename
 267752    20    1740  269512  41cc8 STM32746G_DISCOVERY.elf
Finished building: default.size.stdout

12:08:25 Build Finished. 0 errors, 0 warnings. (took 364ms)
```

Projekt auf das Board aufladen:



Alle Parameter wie voreingestellt dabei belassen.

Auf dem Bildschirm soll ein Bild von einer Frau mit Blumen erscheinen.

In dem Beispiel ist das LED1 verwendet, um ein Fehler in der Konfiguration zu signalisieren:

```
265 static void Error_Handler(void)
266 {
267     /* Turn LED1 on */
268     BSP_LED_On(LED1);
269     while(1)
270     {
271     }
272 }
```

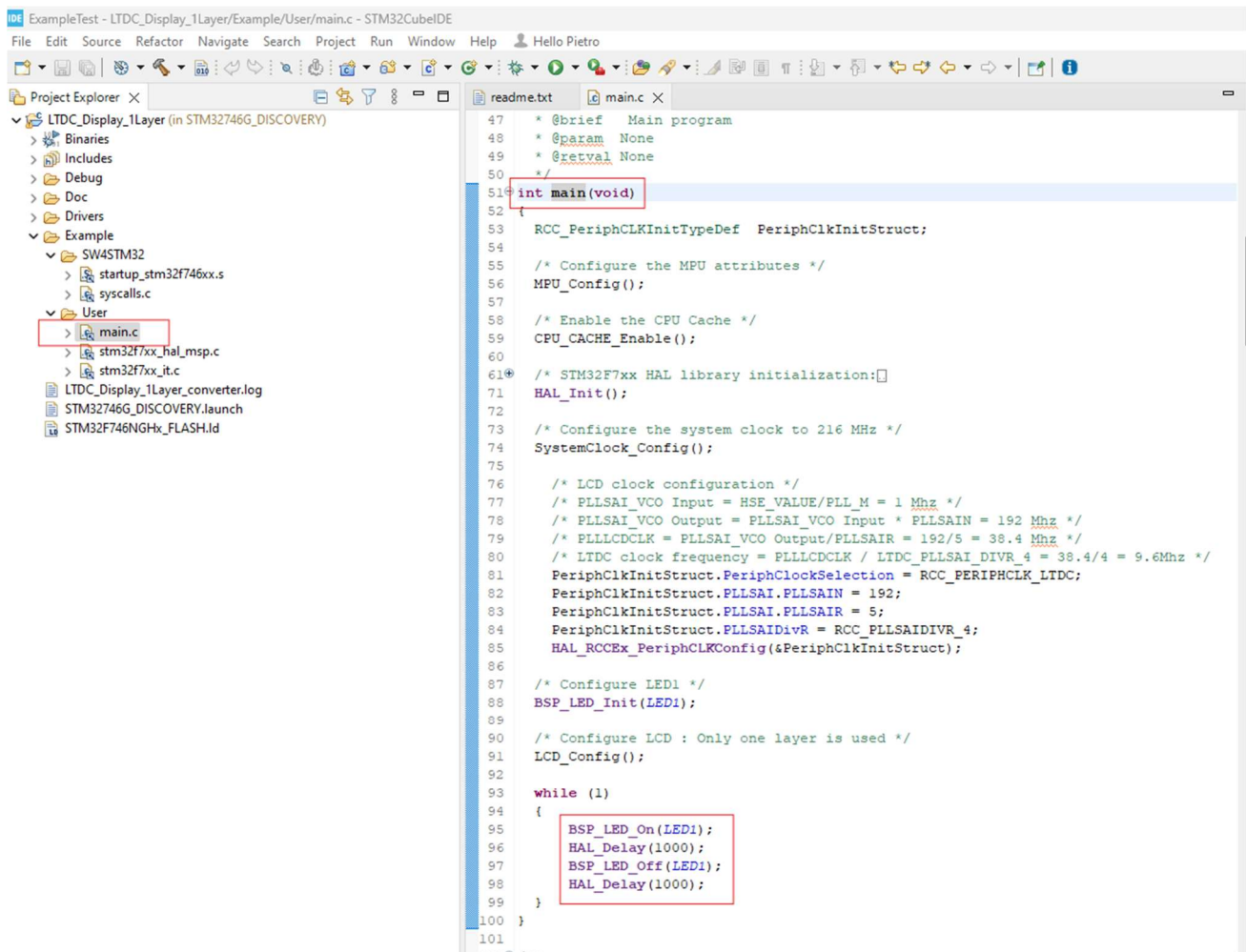
Man kann aber auch das LED1 ein- und ausschalten.

Dafür kann man in dem main.c in der unendlichen Loop folgende Codezeilen hinzufügen:

```
BSP_LED_On(LED1);
HAL_Delay(1000);
BSP_LED_Off(LED1);
HAL_Delay(1000);

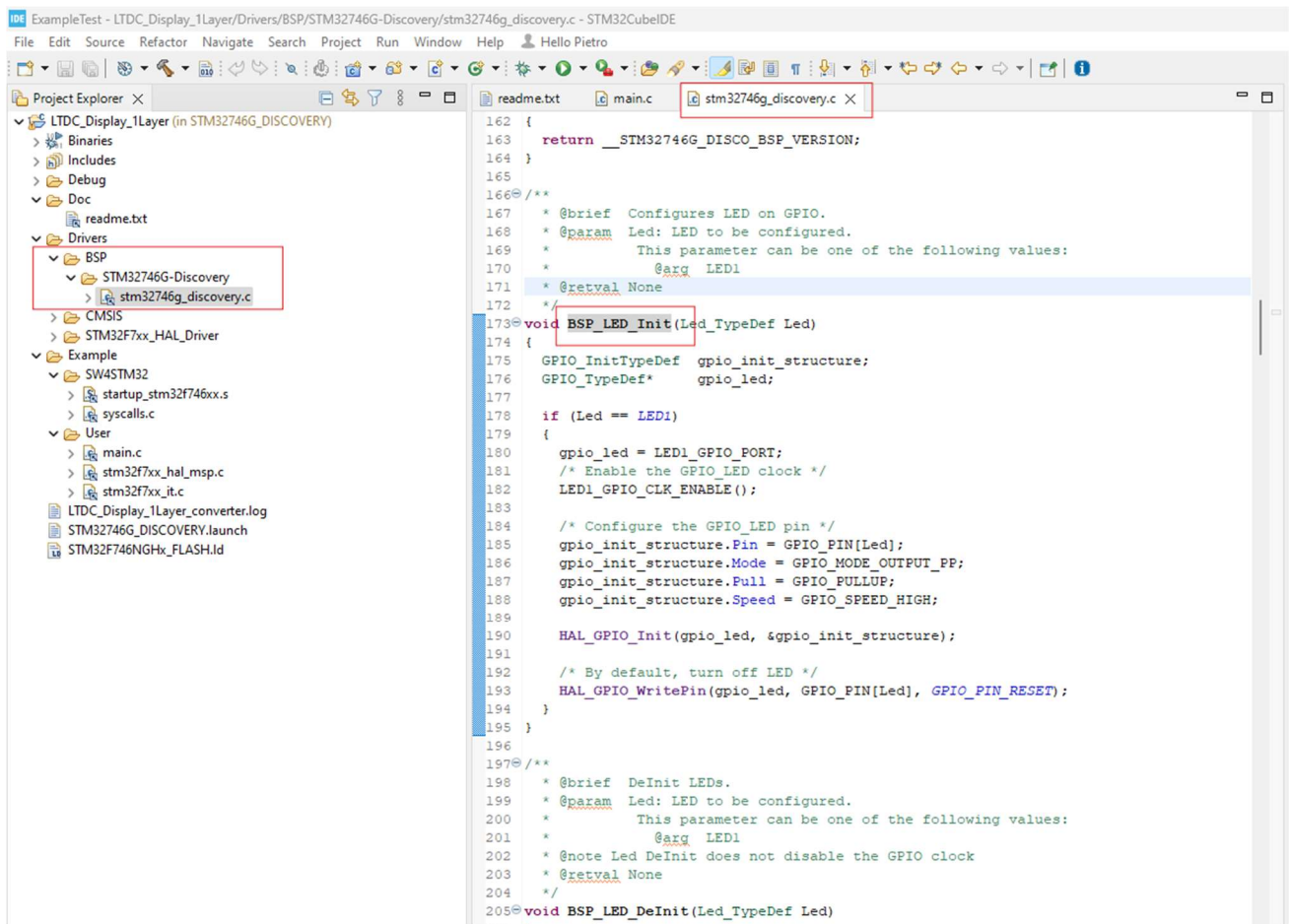
while (1)
{
    BSP_LED_On(LED1);
    HAL_Delay(1000);
    BSP_LED_Off(LED1);
    HAL_Delay(1000);
}
```


Hier die globale Sicht:



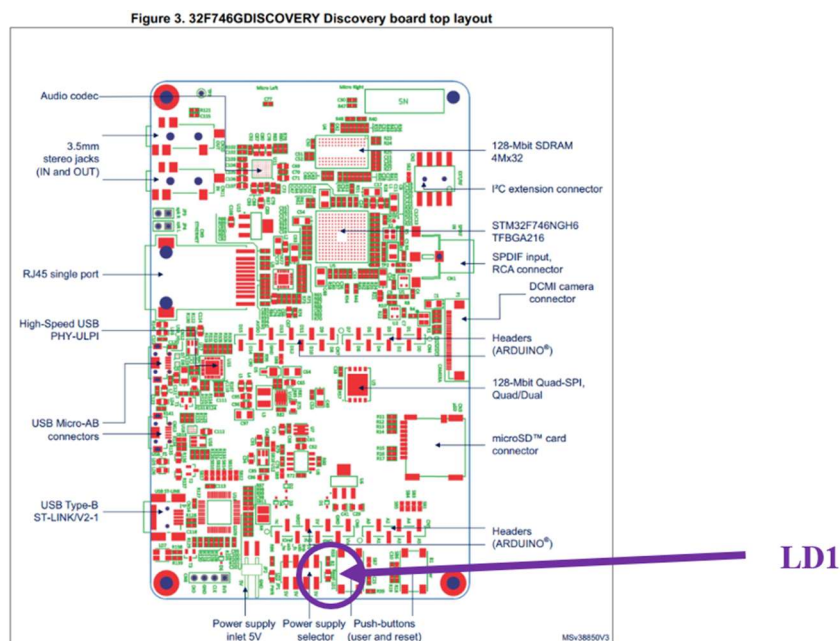
Für diese Implementierung wurden Befehle aus dem Board Support Package (BSP) verwendet. In den nächsten Kapiteln wird gezeigt, wie man die Funktionalität von LED toggeln nur mit HAL-Befehlen implementieren kann.

Die Definition der BSP-Befehle für LEDs ist in dem folgenden Modul zu finden:
„stm32746g_discovery.c“ in dem Ordner Drivers\BSP\ STM32746G_Discovery\

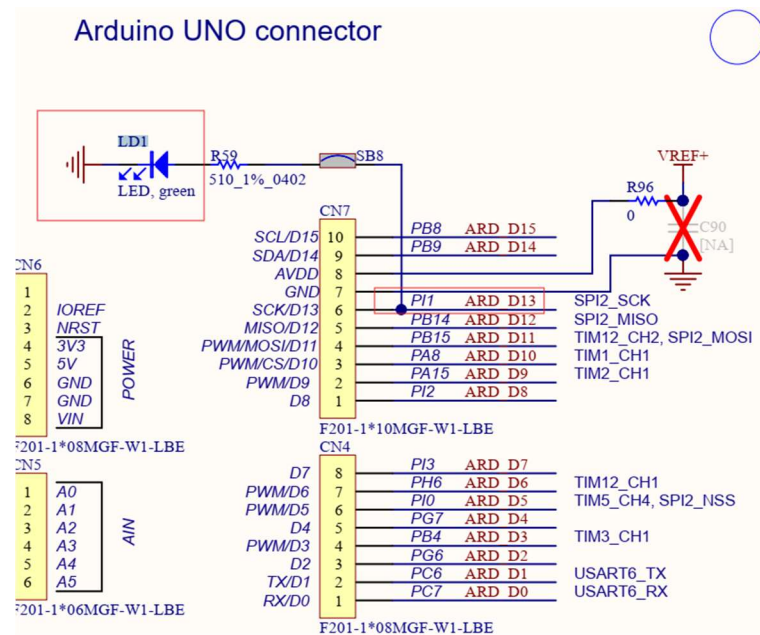


3.5 LED1 Projekt

Das Ziel dieses Projektes ist, die LED1 zu steuern.
Die LED1 ist auf dem Board mit dem Label LD1 gekennzeichnet.

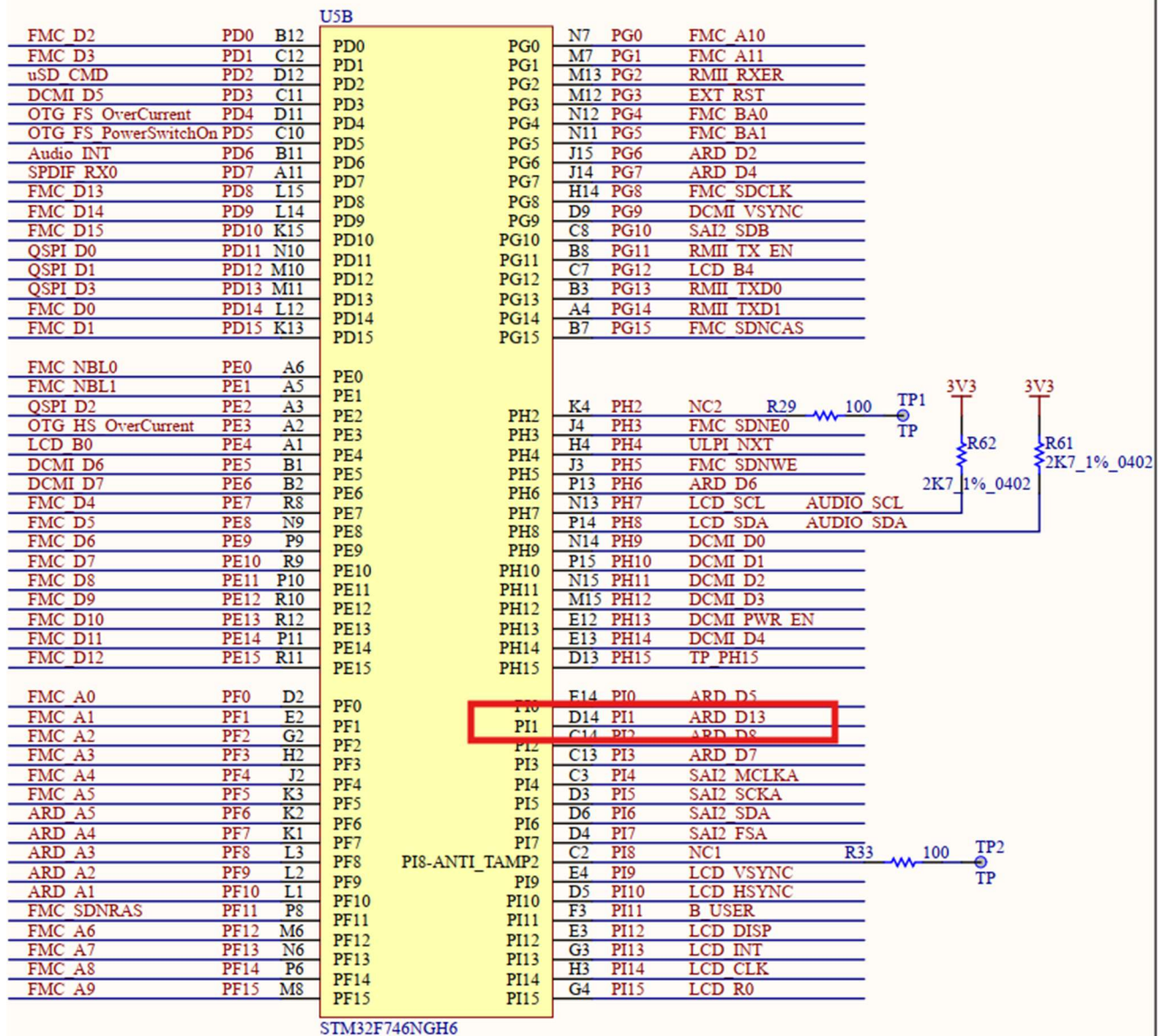


In dem Schematic für die verwendete HW-Revision (im Beispiel C01) sucht man „LD1“, um zu schauen, zu welche Peripherie des Mikrokontrollers angeschlossen ist.



LD1 ist ein LED mit Farbe Grün. Das LED ist durch den Widerstand R59 und der Brücke SB8 an dem PIN PI1 angeschlossen.

Man sucht in den Schematic nach PI1.



In der Datei „stm32f746ng_Product_Specification.pdf“ nach PI1 suchen:

Pinouts and pin description STM32F745xx STM32F746xx

Pin Number							Pin name (function after reset) ⁽¹⁾	Pin type	I/O structure	Notes	Alternate functions	Additional functions	
LQFP100	TFBGA100	WLCSP143	LQFP144	UFBGA176	LQFP176	LQFP208							
-	-	-	-	D14	132	155	D14	PI1	I/O	FT	-	TIM8_BKIN2, SPI2_SCK/I2S2_CK, FMC_D25, DCMI_D8, LCD_G6, EVENTOUT	-

Das Mikrokontroller Pin ist **D14** (Name **PI1**) und kann als Input oder Output (Pin type: I/O) verwendet werden. Zusätzliche Funktionen könne zu dem Pin PI konfiguriert werden (TIM8_ ...).

Table 12. STM32F745xx and STM32F746xx alternate function mapping (continued)

Port		AF0	AF1	AF2	AF3	AF4	AF5	AF6	AF7	AF8	AF9	AF10	AF11	AF12	AF13	AF14	AF15
		SYS	TIM1/2	TIM3/4/5	TIM8/9/10/11/LPTIM1/CEC	I2C1/2/3/4/CEC	SPI1/2/3/4/5/6	SPI3/SAI1	SPI2/3/I2S1/2/3/UART5/SPDIFRX	SAI2/USART6/UART4/5/7/8/SPDIFRX	CAN1/2/TIM12/13/14/QUADSPI/LCD	SAI2/QUADSPI/OTG_HS/OTG1_FS	ETH/OTG1_FS	FMC/SDMMC1/OTG2_FS	DCMI	LCD	SYS
Port I	PI0	-	-	TIM5_CH4	-	-	SPI2_NSS/I2S2_WS	-	-	-	-	-	-	FMC_D2_4	DCMI_D13	LCD_G5	EVEN TOUT
	PI1	-	-	-	TIM8_BKIN2	-	SPI2_SCK/I2S2_CK	-	-	-	-	-	-	FMC_D2_5	DCMI_D8	LCD_G6	EVEN TOUT
	PI2	-	-	-	TIM8_CH4	-	SPI2_MISO	-	-	-	-	-	-	FMC_D2_6	DCMI_D9	LCD_G7	EVEN TOUT
	PI3	-	-	-	TIM8_ETR	-	SPI2_MOSI/I2S2_SD	-	-	-	-	-	-	FMC_D2_7	DCMI_D10	-	EVEN TOUT
	PI4	-	-	-	TIM8_BKIN	-	-	-	-	-	-	SAI2_MCK_A	-	FMC_NB_L2	DCMI_D5	LCD_B4	EVEN TOUT
	PI5	-	-	-	TIM8_CH1	-	-	-	-	-	-	SAI2_SCK_A	-	FMC_NB_L3	DCMI_VSYNC	LCD_B5	EVEN TOUT
	PI6	-	-	-	TIM8_CH2	-	-	-	-	-	-	SAI2_SD_A	-	FMC_D2_8	DCMI_D6	LCD_B6	EVEN TOUT

PI1 ist Teil von Port I.

Somit ist PI1 auch Teil von GPIOI.

STM32F745xx STM32F746xx

Memory mapping

Table 13. STM32F745xx and STM32F746xx register boundary addresses (continued)

Bus	Boundary address	Peripheral
	0x4002 2000 - 0x4002 23FF	GPIOI

In dem Dokument „rm0385-stm32f75xxx-and-stm32f74xxx-advanced-armbased-32bit-mcus-stmicroelectronics.pdf“ sind die Informationen zum GPIO enthalten.

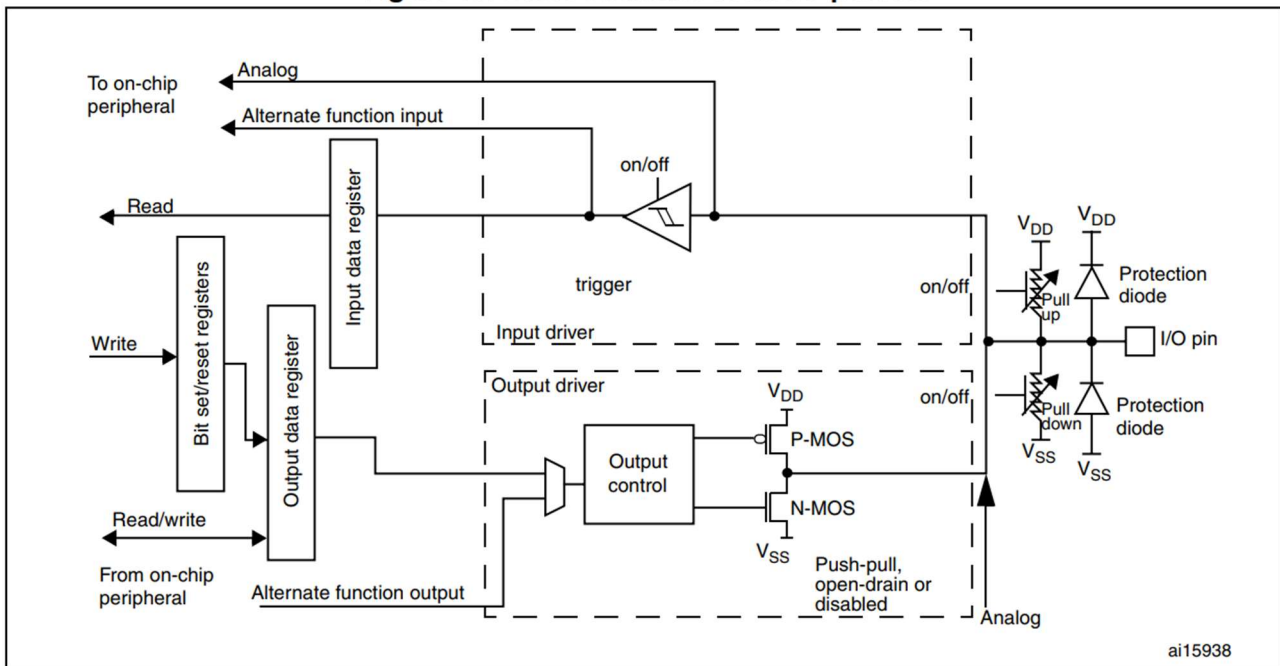
6.3 GPIO functional description

Subject to the specific hardware characteristics of each I/O port listed in the datasheet, each port bit of the general-purpose I/O (GPIO) ports can be individually configured by software in several modes:

- Input floating
- Input pull-up
- Input-pull-down
- Analog
- Output open-drain with pull-up or pull-down capability
- Output push-pull with pull-up or pull-down capability
- Alternate function push-pull with pull-up or pull-down capability
- Alternate function open-drain with pull-up or pull-down capability

Each I/O port bit is freely programmable, however the I/O port registers have to be accessed as 32-bit words, half-words or bytes. The purpose of the GPIOx_BSRR and GPIOx_BRR registers is to allow atomic read/modify accesses to any of the GPIOx_ODR registers. In this way, there is no risk of an IRQ occurring between the read and the modify access.

Figure 17. Basic structure of an I/O port bit



Damit die LED LD1 eingeschaltet werden kann, muss der Pin P1 (D14) des Mikrokontrollers als GPIOI Funktion konfiguriert werden:

- Als Output
- Pull-up

In dem Beispiel für das Testen der Programmierungsumgebung haben wir die BSP-Befehle für LED gesehen. Dort findet man folgende Konfiguration:

```

172  /*
173  void BSP_LED_Init(Led_TypeDef Led)
174  {
175      GPIO_InitTypeDef  gpio_init_structure;
176      GPIO_TypeDef*      gpio_led;
177
178      if (Led == LED1)
179      {
180          gpio_led = LED1_GPIO_PORT;
181          /* Enable the GPIO_LED clock */
182          LED1_GPIO_CLK_ENABLE();
183
184          /* Configure the GPIO_LED pin */
185          gpio_init_structure.Pin = GPIO_PIN[Led];
186          gpio_init_structure.Mode = GPIO_MODE_OUTPUT_PP;
187          gpio_init_structure.Pull = GPIO_PULLUP;
188          gpio_init_structure.Speed = GPIO_SPEED_HIGH;
189
190          HAL_GPIO_Init(gpio_led, &gpio_init_structure);
191
192          /* By default, turn off LED */
193          HAL_GPIO_WritePin(gpio_led, GPIO_PIN[Led], GPIO_PIN_RESET);
194      }
195  }

```

Wobei:

```

#define LED1_GPIO_PORT      GPIOI
const uint32_t GPIO_PIN[LEDn] = {LED1_PIN};
#define LED1_PIN            GPIO_PIN_1

```

Der Befehl, um auf dem Pin zu schreiben ist wie folgt definiert:

```
/**
 * @brief Sets or clears the selected data port bit.
 *
 * @note This function uses GPIOx_BSRR register to allow atomic read/modify
 *       accesses. In this way, there is no risk of an IRQ occurring between
 *       the read and the modify access.
 *
 * @param GPIOx where x can be (A..K) to select the GPIO peripheral.
 * @param GPIO_Pin specifies the port bit to be written.
 *       This parameter can be any combination of GPIO_PIN_x where x can be (0..15).
 * @param PinState specifies the value to be written to the selected bit.
 *       This parameter can be one of the GPIO_PinState enum values:
 *           @arg GPIO_PIN_RESET: to clear the port pin
 *           @arg GPIO_PIN_SET: to set the port pin
 * @retval None
 */
void HAL_GPIO_WritePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin, GPIO_PinState PinState)
```

Wenn man kein BSP verwendet, dann muss man folgender Befehl schreiben:

```
HAL_GPIO_WritePin(GPIOI, GPIO_PIN_1, GPIO_PIN_SET);
```

Für das Toggeln von dem LED1 braucht man dann folgende Befehle:

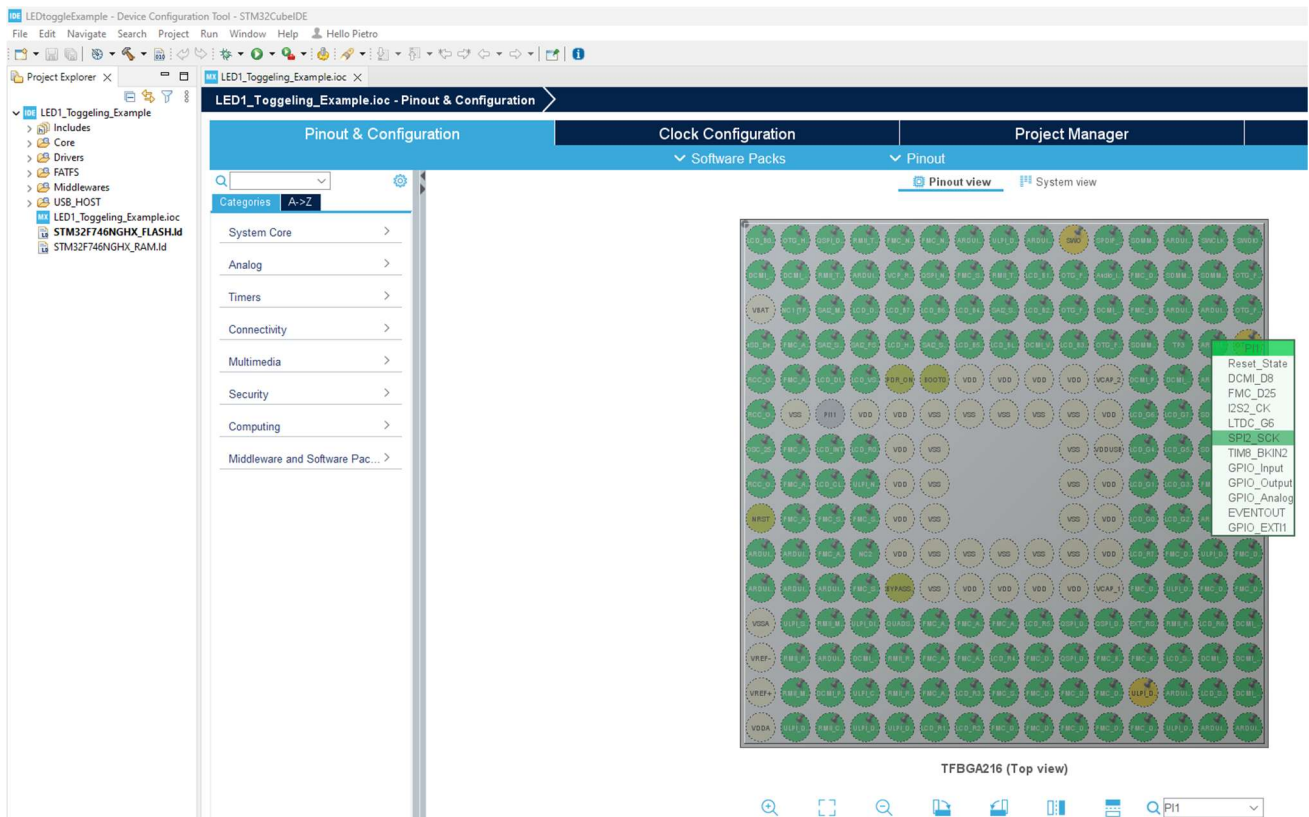
```
HAL_GPIO_WritePin(GPIOI, GPIO_PIN_1, GPIO_PIN_SET); // Einschalten
HAL_Delay(1000); // Status halten für 1 s
HAL_GPIO_WritePin(GPIOI, GPIO_PIN_1, GPIO_PIN_RESET); // Ausschalten
HAL_Delay(1000); // Status halten für 1 s
```

Die Konfiguration des Pins PI1 kann als code oder mit der interaktiven visuellen Umgebung generiert werden.

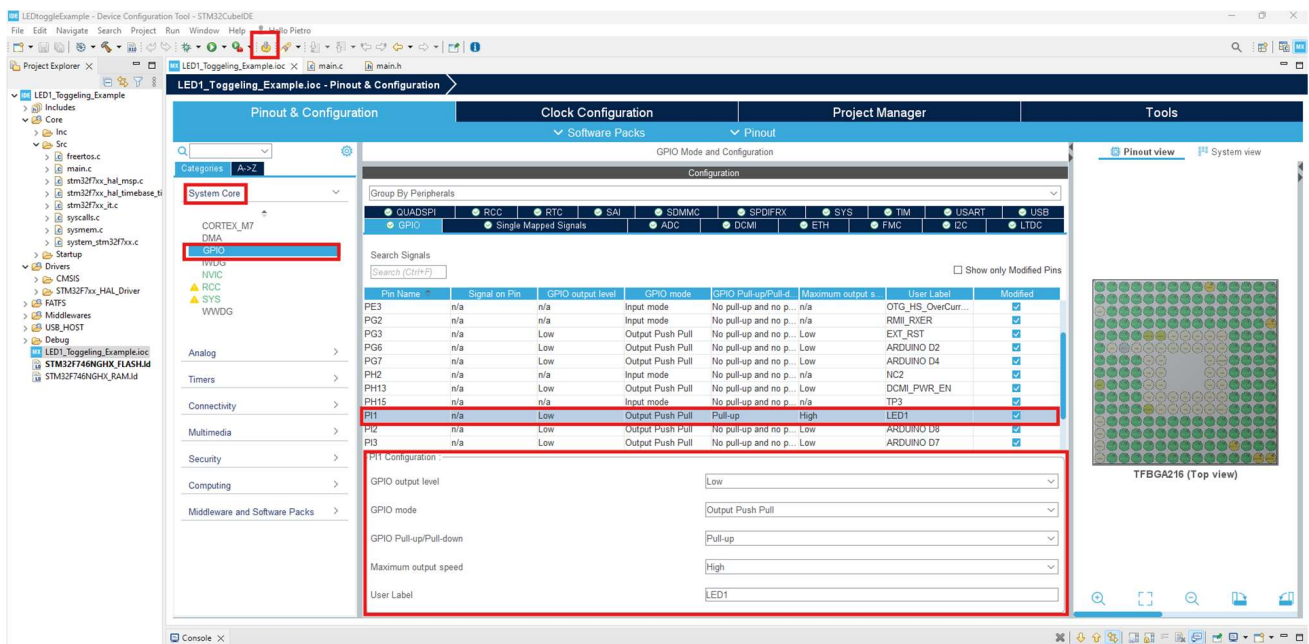
In dem STM32CubeIDE ein neues Projekt starten, mit dem Board Selector das verwendete Board wählen und alle Default Einstellungen beibehalten.

Beim Pinout & Configuration in der Suchmaske unten rechts PI1 eingeben.

Das Pin PI1 wird auf dem Mikrokontroller blinken. Auf dem blinkenden PIN klicken und auf GPIO_Output setzen:



Danach auf System Core klicken und GPIO wählen und entsprechend Bild hier unten konfigurieren:



Danach auf „Device Configuration Tool Code Generation“ klicken .

[...]

3.6 X-CUBE-TouchGFX installieren

Um X-CUBE-TouchGFX zu installieren, klickt man auf der Webseite [32F746GDISCOVERY - Discovery kit with STM32F746NG MCU - STMicroelectronics](#) in dem Bereich “Tools & Software” auf X-CUBE-TouchGFX.

MCU and MPU embedded software

Part number	Status	Description	Type	Supplier
STM32CubeF7	ACTIVE	STM32Cube MCU Package for STM32F7 series (HAL, Low-Layer APIs and CMSIS, USB, TCP/IP, File system, RTOS, Graphic - and examples running on ST boards)	STM32Cube MCU & MPU Packages	ST
X-CUBE-AZRTOS-F7	ACTIVE	Azure RTOS software expansion for STM32Cube for STM32F7 series	STM32Cube Expansion Packages	ST
X-CUBE-TOUCHGFX	ACTIVE	TouchGFX advanced and free of charge graphical framework optimized for STM32 microcontrollers	STM32Cube Expansion Packages	ST

Eine Zip Datei wird heruntergeladen (man muss by MyST angemeldet sein), dann entpackt und installiert.

In dem entpackten Ordner in dem Ordner Utilities\PC_Software\TouchGFXDesigner die .exe Datei starten.

3.6.1 Install Programmer

Der Programmer für die Programmierung/Debug-Schnittstelle muss extra installiert werden, damit man vom TouchGFXDesigner die GUI auf das Board aufladen kann.

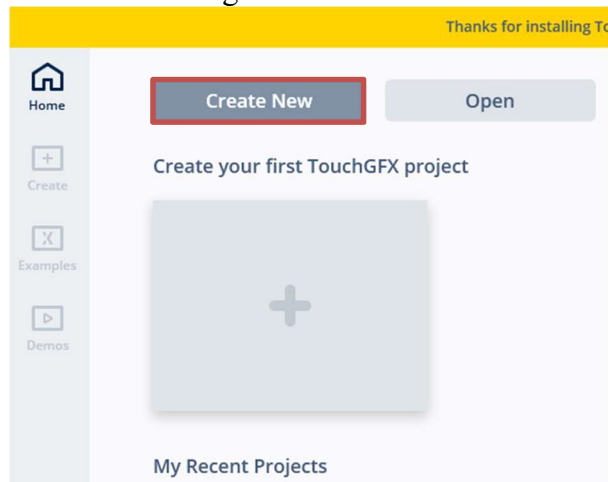
Dafür auf der Webseite [STM32CubeProg - STM32CubeProgrammer software for all STM32 - STMicroelectronics](#) im Bereich „Get Software“ die Version des Programmes entsprechend Betriebssystem herunterladen, entpacken und ausführen (Default Einstellungen beibehalten, damit GFX den Programmer finden kann).

Get Software

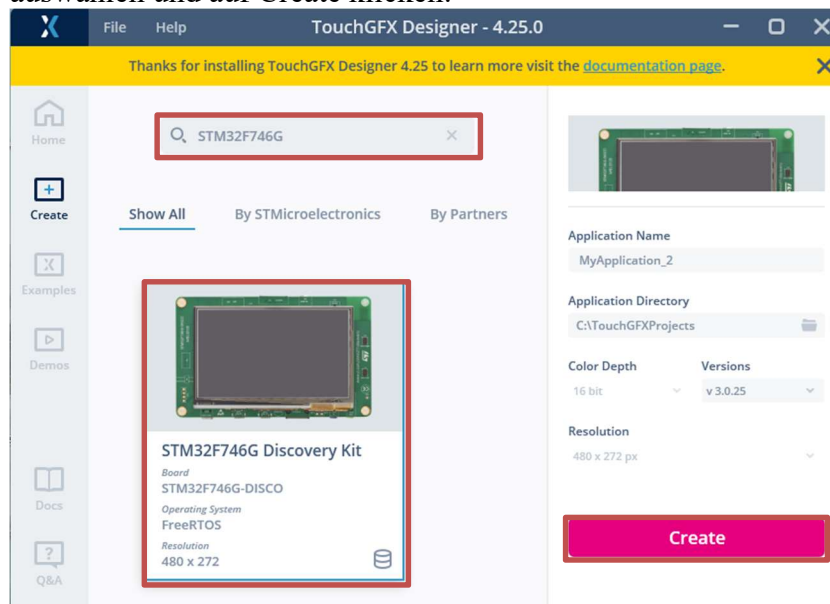
Part Number	General Description	Latest version	Download	All versions
+ STM32CubePrg-Lin	STM32CubeProgrammer software for Linux	2.17.0	Get latest	Select version
+ STM32CubePrg-Mac	STM32CubeProgrammer software for Mac	2.17.0	Get latest	Select version
+ STM32CubePrg-W32	STM32CubeProgrammer software for Win32	2.17.0	Get latest	Select version
+ STM32CubePrg-W64	STM32CubeProgrammer software for Win64	2.17.0	Get latest	Select version

3.7 X-CUBE-TouchGFX starten und ein erstes Projekt entwickeln

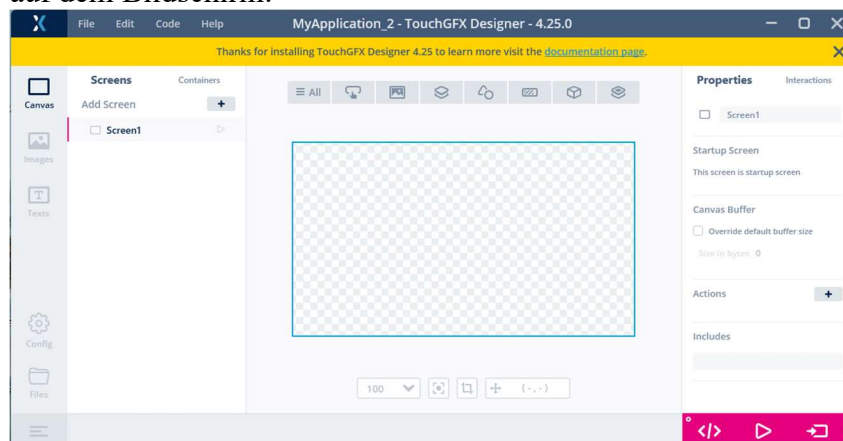
TouchGFX Designer starten und auf Create New klicken:



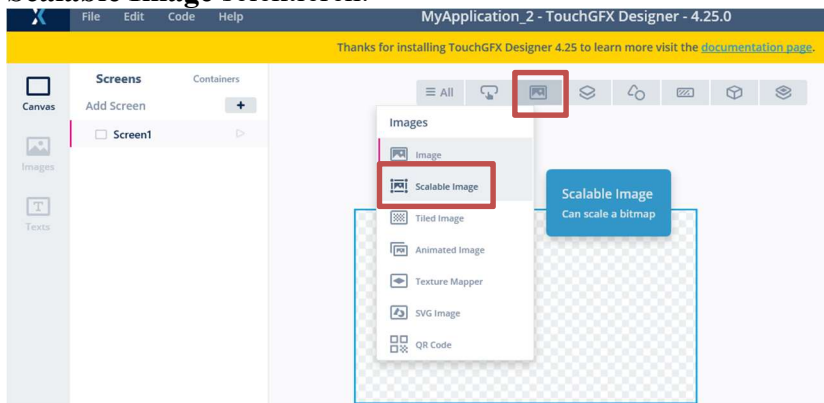
In der Search-Maske das Board STM32F746G angeben und dann STM32F746G Discovery Kit auswählen und auf Create klicken:



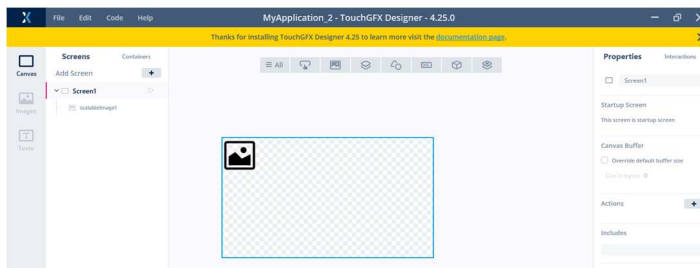
Das Projekt zeigt den Bildschirm vom Board und einige Werkzeuge für die Gestaltung der Objekten auf dem Bildschirm:



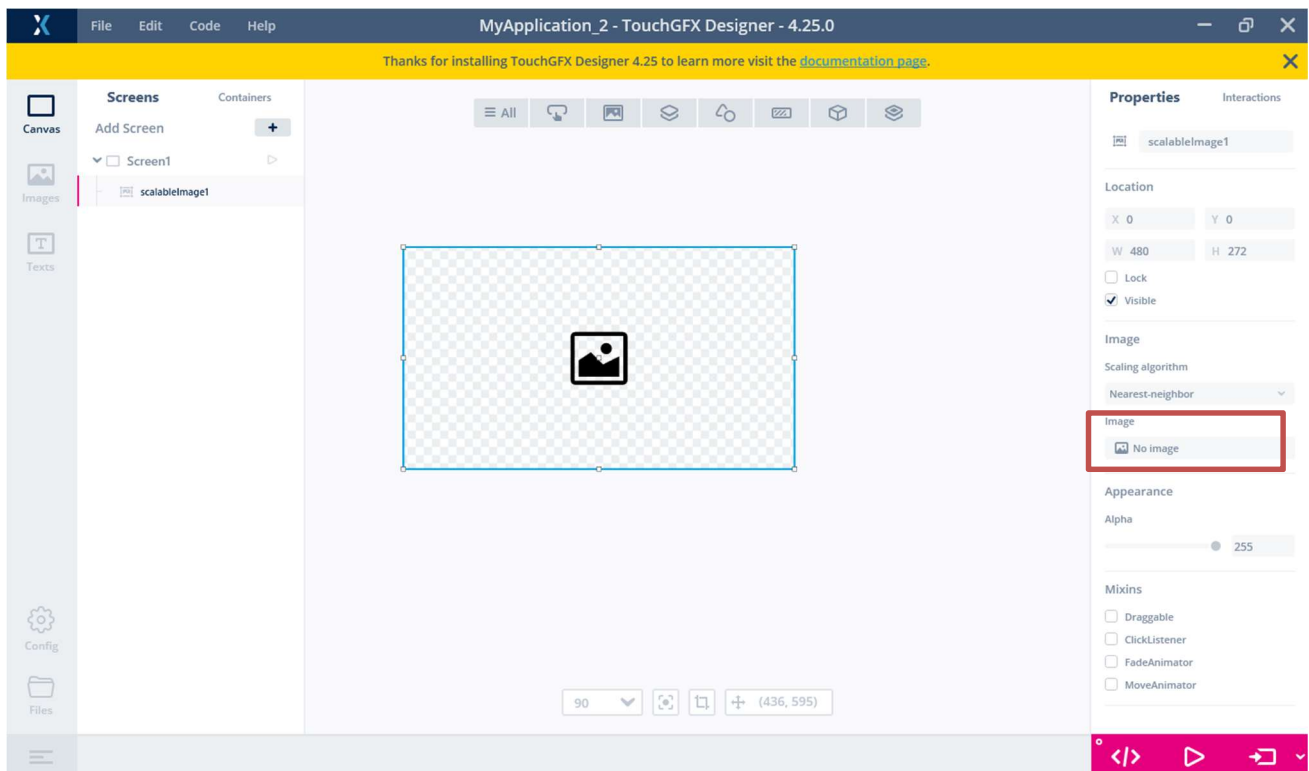
Auf dem Symbol **Images** klicken, um ein Hintergrundbild für den Bildschirm zu wählen. Dazu **Scalable Image** selektieren:

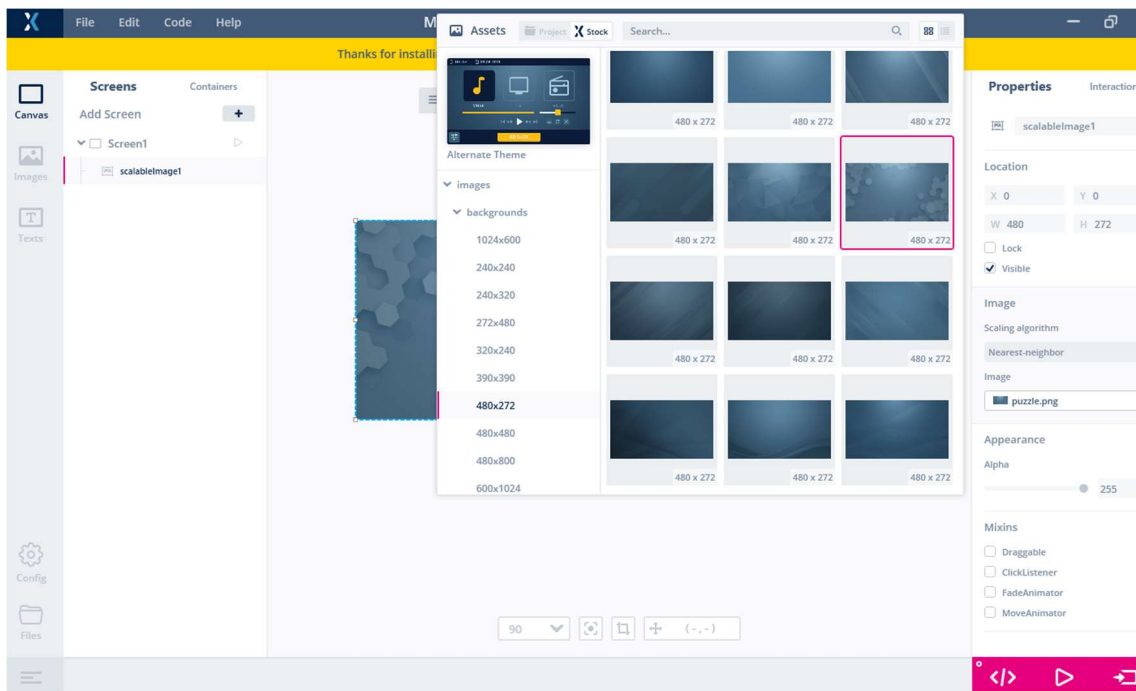


Ein Platzhalter für ein Bild erscheint auf dem Bildschirm:

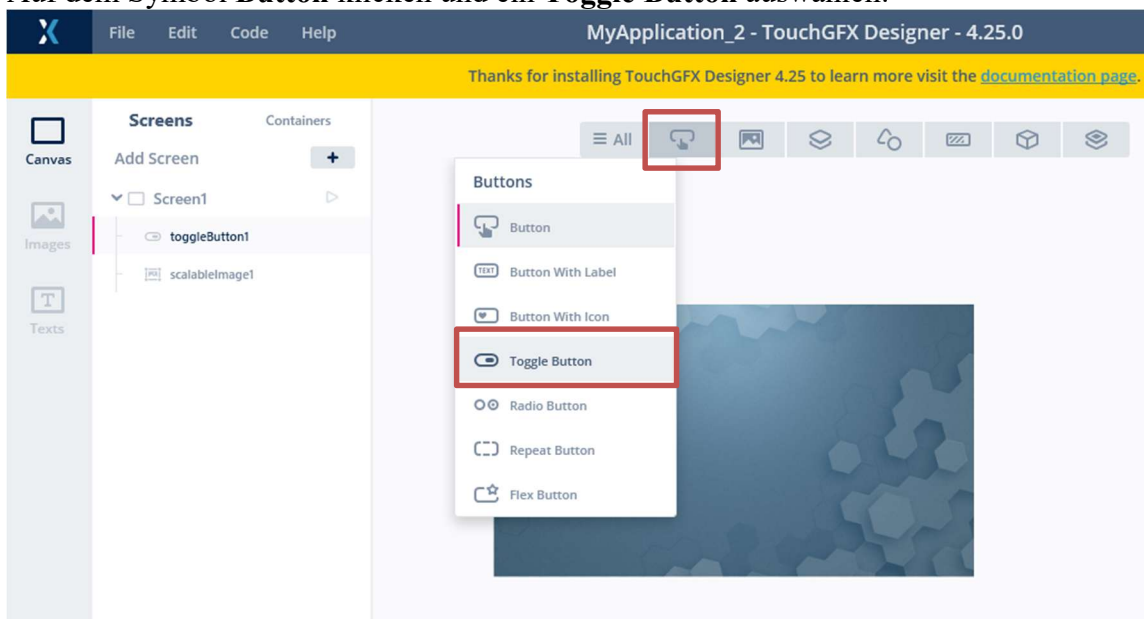


Bitte den Platzhalter auf die Größe des Bildschirmes skalieren. In dem Menu **Image** auf der rechten Seite ein Bild durch den Browser auswählen:

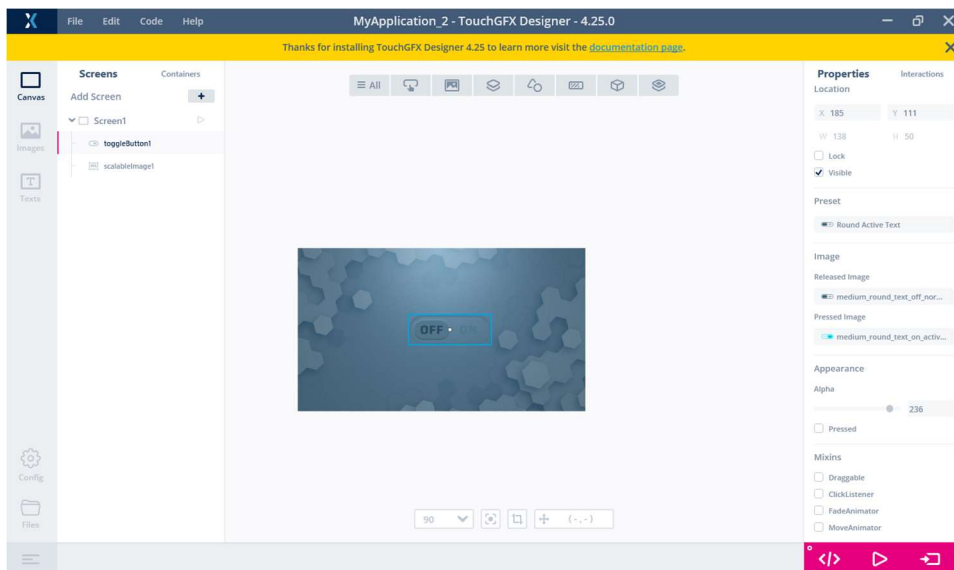




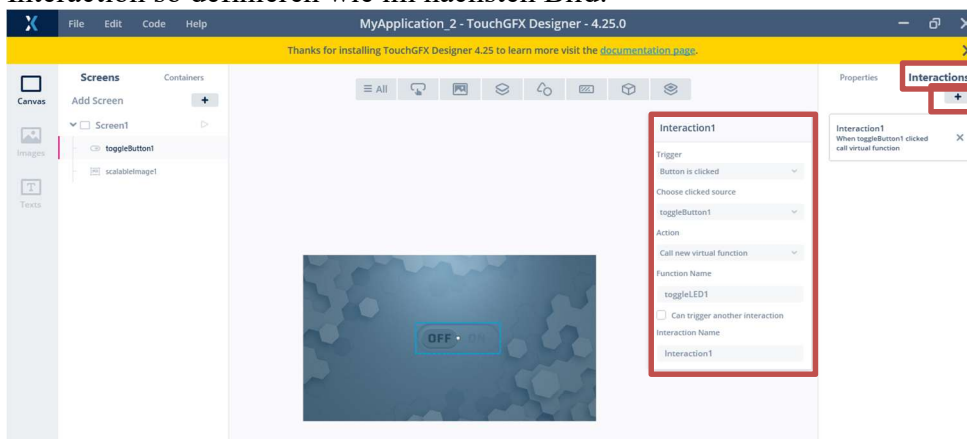
Auf dem Symbol **Button** klicken und ein **Toggle Button** auswählen:



Ein Platzhalter für den Toggle Button erscheint und kann z.B. in der Mitte des Bildschirms platziert werden. Das Aussehen vom Button kann in den Menüs an der rechten Seite beeinflusst werden:



Der Schalter hat den Name **toggleButton1**. Der Bildschirm heißt **Screen1**.
In dem Bereich Interactions kann die Interaktion mit dem Schalter definiert werden. Bitte die Interaction so definieren wie im nächsten Bild:

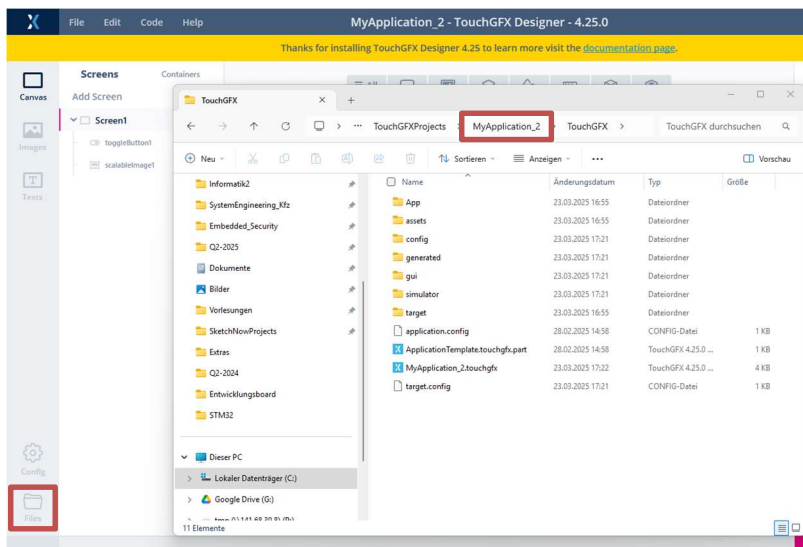


Beim Klicken soll eine Funktion **toggleLED1** aufgerufen werden. Diese Funktion wird später definiert.

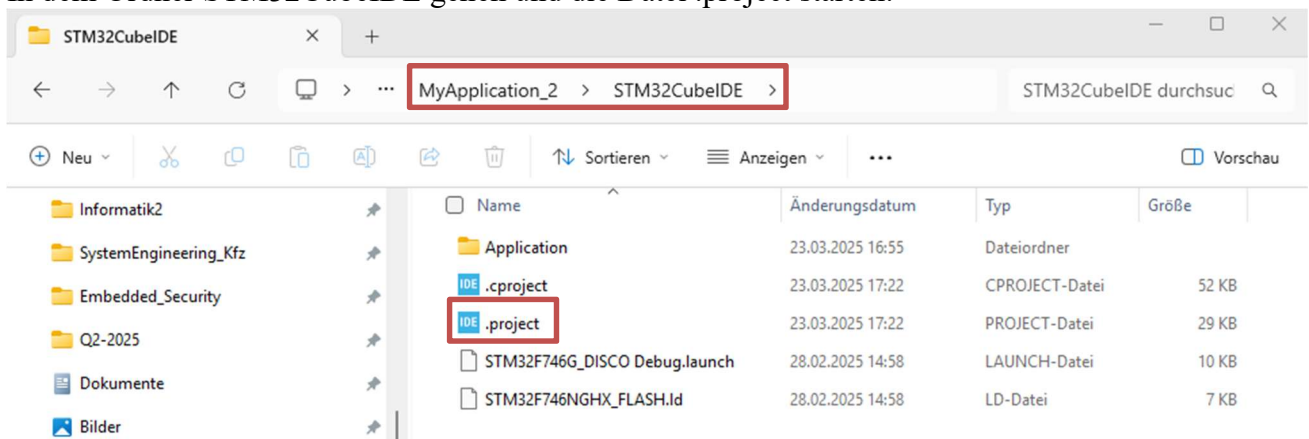
Jetzt kann das Projekt generiert werden, dafür klicken Sie auf „Generate Code“  unten rechts.

Nach der erfolgreichen Generierung erscheint unten links:  **Generate Code** | Done

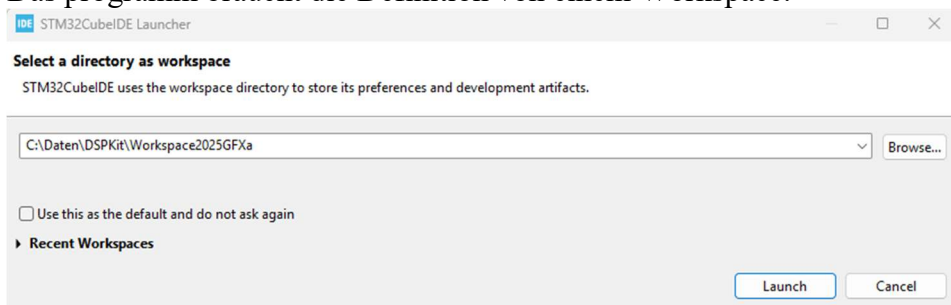
Jetzt auf **Files** unten links klicken, damit geht der File Browser an und man kann den Ordner einer Hierarchie höher wählen.



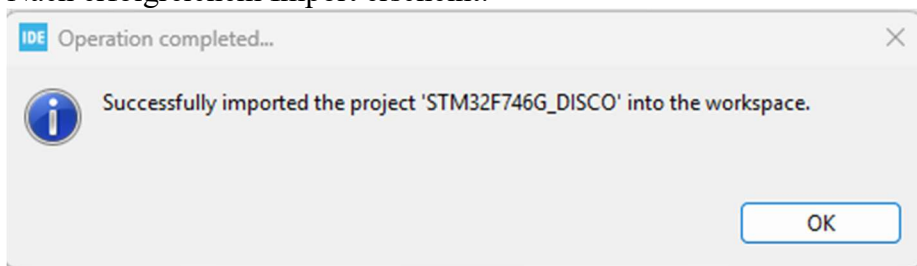
In dem Ordner STM32CubeIDE gehen und die Datei .project starten:



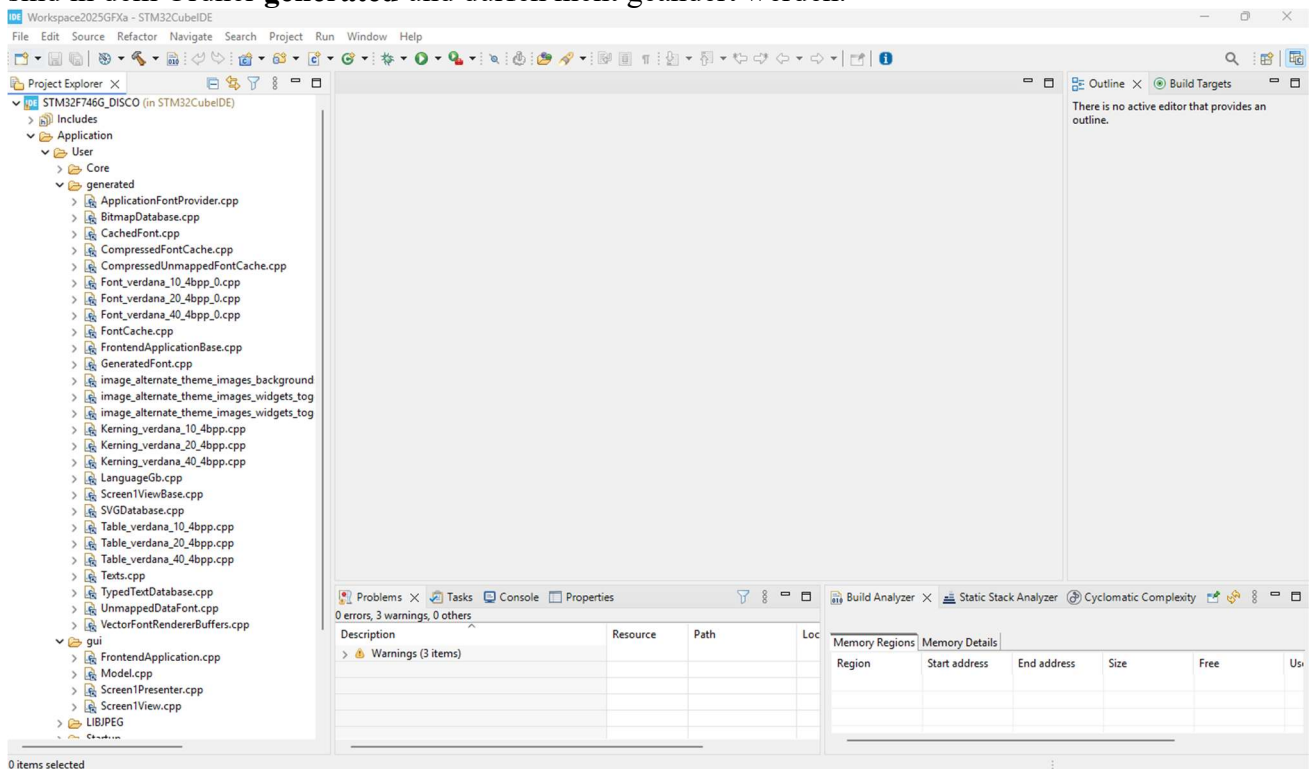
Das Programm braucht die Definition von einem Workspace:



Nach erfolgreichem Import erscheint:



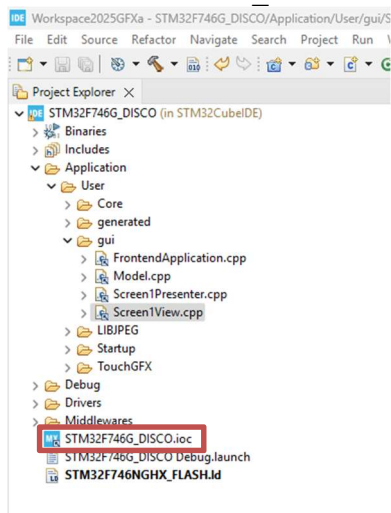
Danach wird die Eclipse Umgebung gestartet und die Projektfiles sind sichtbar. Die generierten Files sind in dem Ordner **generated** und dürfen nicht geändert werden.

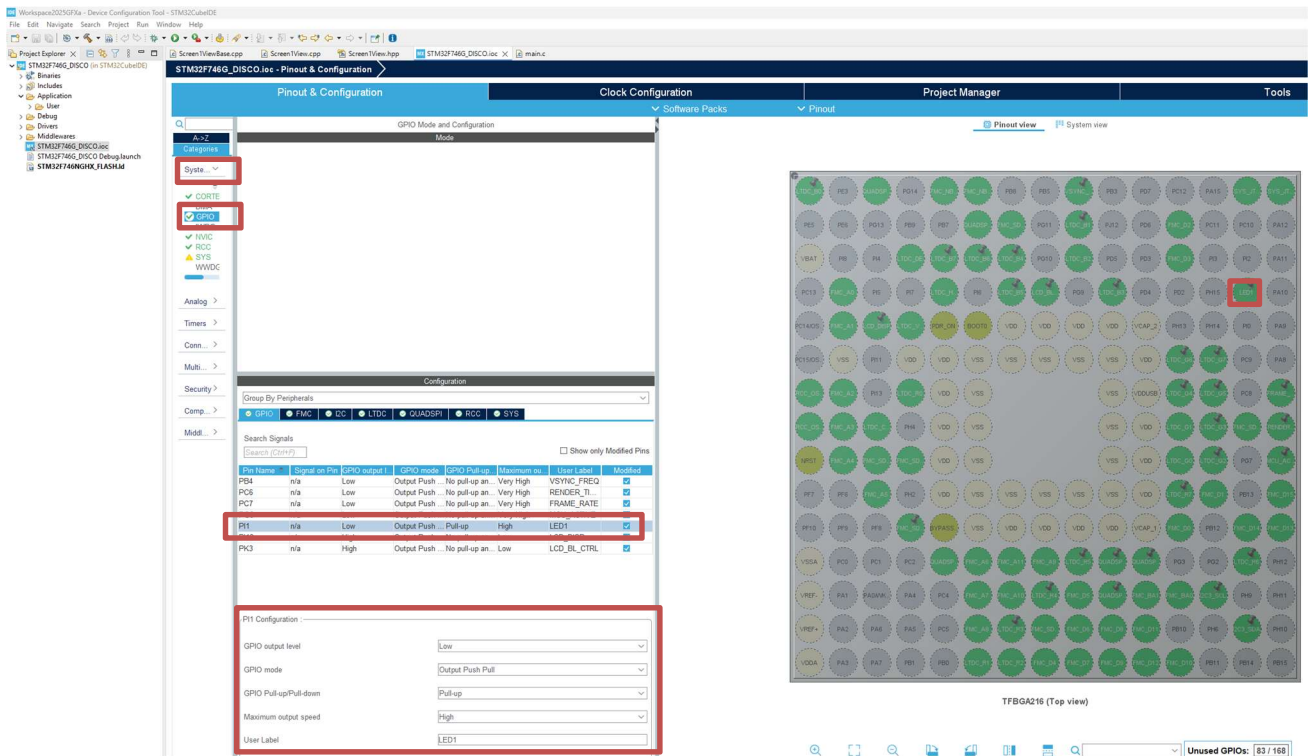


Die Datei, die wir anpassen dürfen befinden sich z.B. im Ordner **gui**.

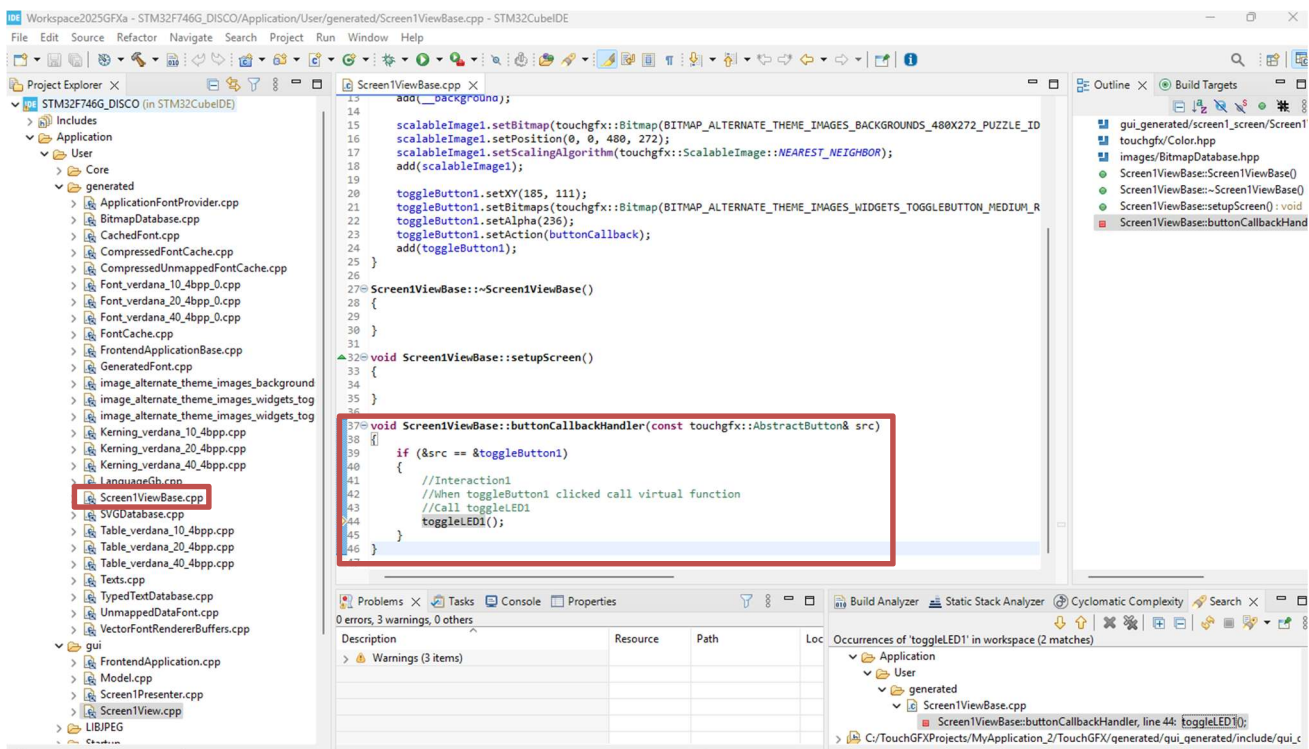
In dem Projekt muss das LED1 in GPIO konfiguriert werden, dafür verwendet man die Vorgehensweise, die in Kapitel **LED1 Projekt** präsentiert wurde.

Auf STM32F746G_DISCO.ioc klicken:





Unsere Funktion **toggleLED1()** ist in der Datei **Screen1ViewBase.cpp** in dem **buttonCallbackHandler** aufgerufen, aber ist noch nicht definiert worden:

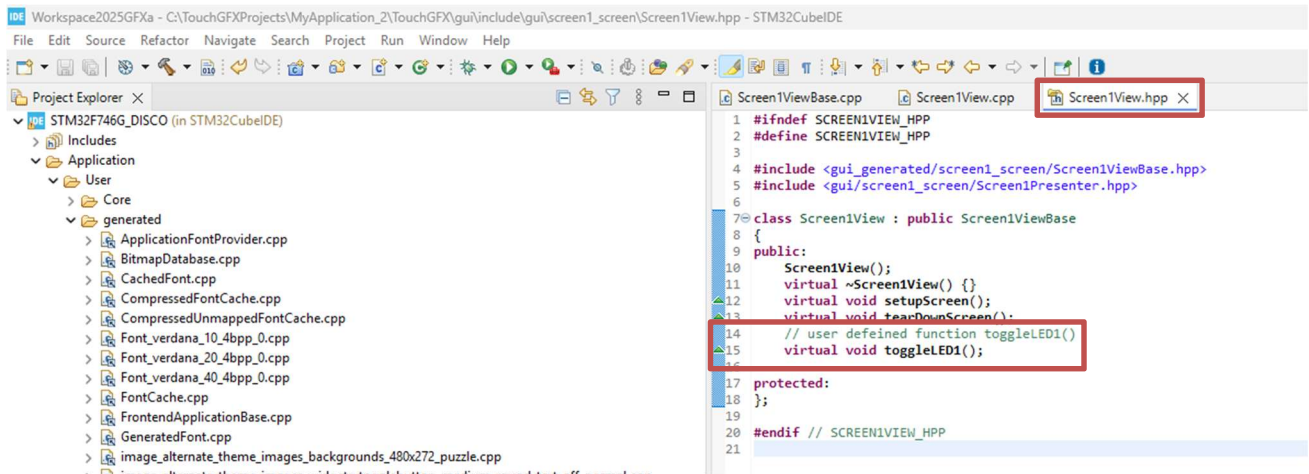


Die Funktion **toggleLED1()** soll in der Klassendeklaration (**Screen1View.hpp**) aufgenommen werden

```

// user defined function toggleLED1()
virtual void toggleLED1();

```

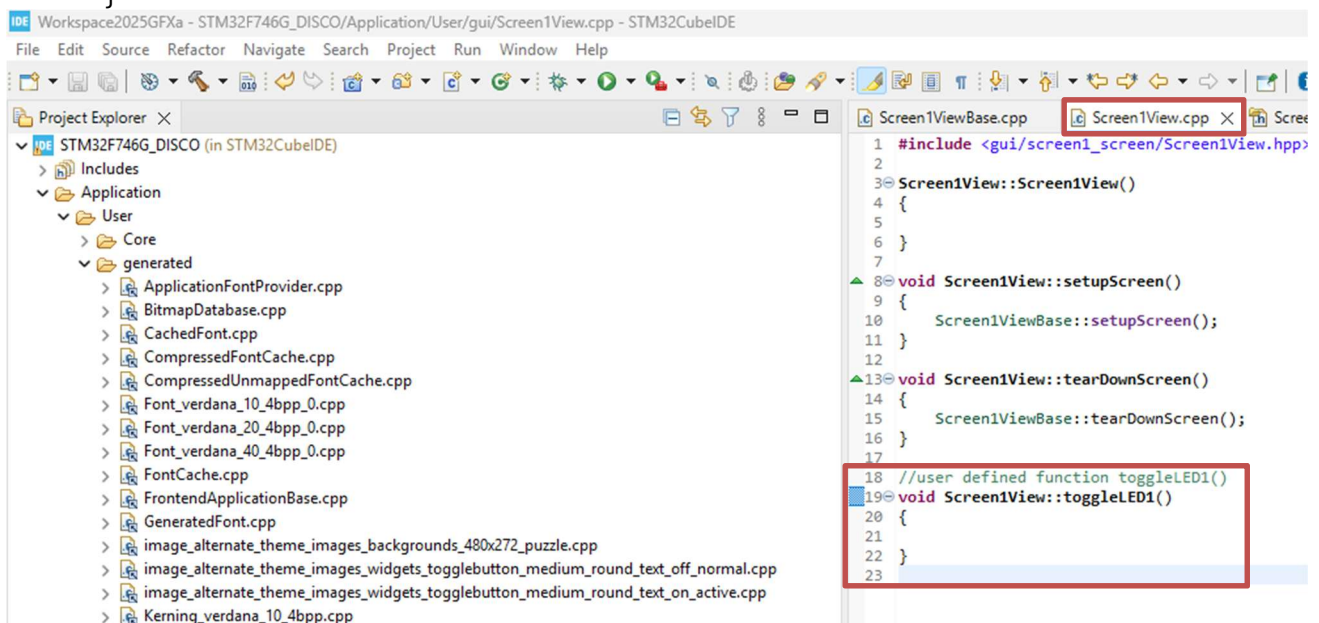


und in der Klassendefinition (Screen1View.cpp) definiert werden:

```

//user defined function toggleLED1()
void Screen1View::toggleLED1()
{
}

```



Files speichern und prüfen, ob das Projekt gebaut werden kann:


```

//user defined function toggleLED1()
void Screen1View::toggleLED1()
{
    if (toggleButton1.getState())
    {
        HAL_GPIO_WritePin(GPIOI, GPIO_PIN_1, GPIO_PIN_SET); // Einschalten
    }
    else
    {
        HAL_GPIO_WritePin(GPIOI, GPIO_PIN_1, GPIO_PIN_RESET);
    }
}
}

```

Jetzt kann das Projekt gebildet werden und auf dem Board hochgeladen werden.